Построение и анализ алгоритмов

Справочник для устного экзамена

Вопросы из четырёх листов файла «вопросы с зачета пива (1).xlsx» сгруппированы по содержанию. К ним добавлены темы из предоставленной программы. Учебные примеры ниже составлены для объяснения: если в отзыве не записаны конкретные строки или рисунок графа, они не выдаются за исходное экзаменационное задание.

Как пользоваться. Сначала прочитайте «Ответ на экзамене» в нужном разделе. Затем воспроизведите пример и доказательство без подсказки. В приложении А найдите исходную ячейку и соответствующие разделы. Формулировки студентов — свидетельства о вопросах, а не источник математической истины. Имена, оценки, бытовые комментарии и три изображения-мема исключены. Инструкции внутри таблицы не использовались как указания к работе.

Обозначения. Для графа n = |V|, m = |E|. Для строк N — длина текста, M — длина одного образца, L — сумма длин набора образцов, z — число выданных вхождений, σ — размер алфавита. log — логарифм с постоянным основанием; ln — натуральный. O*(c^n) скрывает полиномиальный множитель. Взвешенные задачи предполагают явно указанные условия на веса. Память указана дополнительно к входу, если не оговорено иное.

План ответа. Постановка и условия → состояние алгоритма → шаги → инвариант или аргумент корректности → время и память с указанием реализации → короткий пример → граничный случай. Название алгоритма само по себе не заменяет объяснение.

Что особенно часто уточняют в файле. Почему КМП линейный при наличии откатов; как именно перечислять множества для 3-раскраски; что алгоритм делает на K_5; откуда вероятность 2/[n(n−1)] у Каргера; сколько нужно независимых запусков; почему в Каргере–Штейне n/√2; зачем обратное ребро; почему 2-приближение действительно даёт коэффициент 2; чем индекс текста отличается от индекса образцов.

Содержание

1. Сложность и базовые определения 3

2. Поиск с возвращением и ветви и границы 5

3. Метод Монте-Карло и вероятности повторений 6

4. Жадные алгоритмы 9

5. Раскраска графа в три цвета 11

6. Минимальные разрезы: Каргер и Каргер–Штейн 15

7. Задача о коммивояжёре (TSP) 18

8. Кратчайшие пути: BFS, Дейкстра, A*, Theta*, ALT 21

9. Потоки: Форд–Фалкерсон и Гольдберг (push–relabel) 24

10. Изоморфизм графов и алгоритм Ульмана 27

11. Симбиоз ИИ и A* 29

12. Динамические графы и поддержка МОД 31

13. Редакционное расстояние: Левенштейн и Вагнер–Фишер 32

14. Префикс-функция и Кнут–Моррис–Пратт 35

15. Ахо–Корасик: бор, ссылки, автомат 37

16. Суффиксные структуры и Укконен 40

17. Динамическое программирование 44

18. Быстрое повторение и типичные ошибки 46

Приложение А. Указатель исходных записей 49

Приложение Б. Навигация по дополнительным темам программы 53

Приложение В. Литература для сверки и углубления 54

1. Сложность и базовые определения

Статус: определения графов и анализ сложностей — из таблицы; vector, unordered_set и виды сложности — дополнение по программе.

1.1. Что такое вычислительная сложность и размер входа?

Ответ на экзамене. Сложность — расход ресурса как функция размера входа. Нужно назвать ресурс, модель вычисления и то, какие входы рассматриваются: худшие, случайные или последовательность операций. O — верхняя асимптотическая оценка, Ω — нижняя, Θ — точный порядок роста.

В RAM-модели операция над машинным словом стоит O(1). Это не означает O(1) для арифметики с числами произвольной длины. Для графа вход в списках смежности имеет размер Θ(n+m), для матрицы — Θ(n^2). Для целого числа x размер двоичной записи Θ(log(x+1)), а не x: алгоритм O(x) может быть псевдополиномиальным.

Количество операций — абстрактная мера; время зависит ещё от реализации, кэшей и оборудования; память бывает общей и дополнительной. Глубина рекурсии тоже занимает память. Всегда уточняйте, входит ли в оценку хранение результата: выдать z вхождений требует Ω(z) времени.

Пример. Два вложенных цикла с границами i=0..n−1 и j=0..i выполняют 1+2+…+n = n(n+1)/2 действий, значит Θ(n^2). Два последовательных линейных цикла дают Θ(n), а не Θ(n^2).

1.2. Чем отличаются амортизированная, средняя и худшая сложности?

Ответ на экзамене. Худшая — максимум по входам заданного размера. Средняя — математическое ожидание при заданном распределении входов или случайности алгоритма. Амортизированная — гарантированная оценка суммарной стоимости последовательности операций, делённая на их число; вероятности для неё не нужны.

Операция	Типичная оценка	Почему есть оговорка
vector[i]	O(1) в худшем случае	адрес вычисляется напрямую
vector.push_back	O(1) амортизированно, O(n) в худшем	иногда переносится весь буфер
vector.insert в середину	O(n) в худшем	сдвиг элементов даже без перевыделения
unordered_set.find	в среднем O(1), в худшем O(n)	коллизии могут собрать всё в одну корзину
unordered_set.insert	в среднем O(1), в худшем O(n)	коллизии и возможный rehash

Доказательство для vector. При геометрическом увеличении вместимости 1,2,4,8,… суммарное число переносов за k добавлений меньше 1+2+4+…+$2^{\lfloor \log_2 k \rfloor} < 2k$. Ещё k действий — запись новых элементов. Итого O(k), то есть O(1) на добавление амортизированно. Увеличение вместимости каждый раз ровно на 1 дало бы Θ(k^2).

Стандартный интерфейс не обещает конкретно удвоение; важна обеспеченная амортизированная гарантия.

Ловушка. «В среднем» для хеш-таблицы не гарантирует защиту от враждебных ключей и плохой хеш-функции. Даже при хорошем среднем времени отдельный rehash дорогой. Не путайте ожидаемое время поиска с амортизированием изменения размера.

1.3. Граф, цикл, дерево, эйлеров и гамильтонов циклы

Ответ на экзамене. Граф $G=(V,E)$ задаёт вершины и рёбра. Графы бывают ориентированные и неориентированные, взвешенные и невзвешенные; простой граф не содержит петель и кратных рёбер. Путь — последовательность смежных вершин; простой путь не повторяет вершины. Простой цикл замыкается и не повторяет вершины, кроме первой/последней. Дерево — связный неориентированный граф без циклов.

Дерево на n вершинах имеет $n-1$ ребро; между любыми двумя вершинами единственный простой путь. Эквивалентность доказывается удалением листьев или тем, что каждое ребро дерева — мост. Остов охватывает все вершины исходного связного графа.

Эйлеров цикл проходит каждое ребро ровно один раз; вершины могут повторяться. В связном по рёбрам неориентированном графе он существует тогда и только тогда, когда все степени чётны. Гамильтонов цикл проходит каждую вершину ровно один раз и возвращается в начало; он не обязан использовать все рёбра. Для него нет аналогичного простого критерия по чётности степеней.

Пример. В K_4 есть гамильтонов цикл $1\to2\to3\to4\to1$, но нет эйлерова: степени равны 3. В задаче коммивояжёра ищут гамильтонов цикл минимального веса. Полный граф содержит ребро между каждой парой различных вершин; у K_5 десять рёбер и степени 4.

2. Поиск с возвращением и ветви и границы

Статус: методы разработки алгоритмов, backtracking и отсечения встречаются в таблице; ферзи — дополнение по программе.

2.1. Как устроен backtracking на задаче о ферзях?

Ответ на экзамене. Строю частичное решение, пробую допустимое продолжение, рекурсивно продолжаю и отменяю выбор. Если частичное решение уже нарушает ограничение, ни одно его продолжение не подходит: ветвь отсекается.

В задаче n ферзей ставим по одному ферзю в строку. Занятые столбцы c, диагонали r−c и r+c храним в множествах или масках. Проверка кандидата тогда O(1) в подходящей модели.

```
search(r):
    if r == n: выдать решение; return
    for c in 0..n-1:
        if c, r-c, r+c свободны:
            занять их; position[r] = c
            search(r+1)
            освободить их
```

Пример. Для n=4 выбор столбцов по строкам [1,3,0,2] (нумерация с нуля) допустим: все столбцы и обе серии диагоналей различны. Если начать [0,2], в третьей строке нет допустимого продолжения, значит нужно вернуться, снять второго ферзя и попробовать другой столбец.

Сложность. Наивно n^n вариантов размещения по строкам. Запрет одинаковых столбцов оставляет не более n! полных перестановок; перебор всех n столбцов в узлах даёт консервативную оценку O(n·n!), память O(n) плюс вывод. Перечисление только свободных кандидатов уменьшает накладные расходы. Диагонали сильно сокращают фактическое дерево, но не делают эту реализацию полиномиальной. Разные способы подсчёта узлов и проверки дают разные полиномиальные множители: их надо проговаривать.

2.2. Что добавляет метод ветвей и границ?

Ответ на экзамене. Backtracking отсекает недопустимые частичные решения. Ветви и границы дополнительно используют оценку лучшего возможного продолжения. Для минимизации храним стоимость уже найденного полного решения U и нижнюю границу LB для ветви. Если LB ≥ U, улучшения в ней не будет.

Почему корректно. Для любого полного решения внутри отсекаемой ветви cost ≥ LB ≥ U, следовательно оно не лучше имеющегося. Если нужны все оптимальные решения, равенство отсекать нельзя без дополнительного учёта.

Порядок обхода, раннее получение хорошего U, более сильные границы, устранение симметрий и выбор наиболее ограниченной переменной сокращают перебор. Дорогая граница может уменьшить число узлов и увеличить общее время. В худшем случае экспоненциальное или факториальное дерево может сохраниться. Подробные границы для коммивояжёра — в §7.

3. Метод Монте-Карло и вероятности повторений

Статус: площадь, размер дерева, условия применимости и вывод вероятностей — из таблицы; общая схема повторений объединяет несколько вопросов.

3.1. Что нужно знать о задаче, чтобы применять Монте-Карло?

Ответ на экзамене. Нужна вероятностная модель, способ выбирать случайные объекты с известным распределением и вычислять величину, связанную с искомым ответом. Требуется обосновать несмещённость или вероятность успеха, независимость повторений и оценить ошибку. Случайная выборка без связи с ответом ничего не гарантирует.

Для площади нужны известная область-ограничитель конечной площади и проверка принадлежности фигуре; для дерева — возможность генерировать случайный путь и знать вероятности выбора. Конечная дисперсия позволяет оценивать среднеквадратическую ошибку; для сильных концентрационных границ нужны соответствующие условия, например ограниченность независимых величин. Применимость не ограничивается геометрией и не означает, что надо заранее знать искомую площадь.

Монте-Карло допускает ошибку ответа при контролируемых затратах; алгоритм Лас-Вегаса всегда возвращает правильный ответ, но его время случайно. У вероятностной 3-раскраски найденную раскраску можно проверить, а неудачный запуск не доказывает невозможность. У Каргера найденный разрез допустим, но может не быть минимальным.

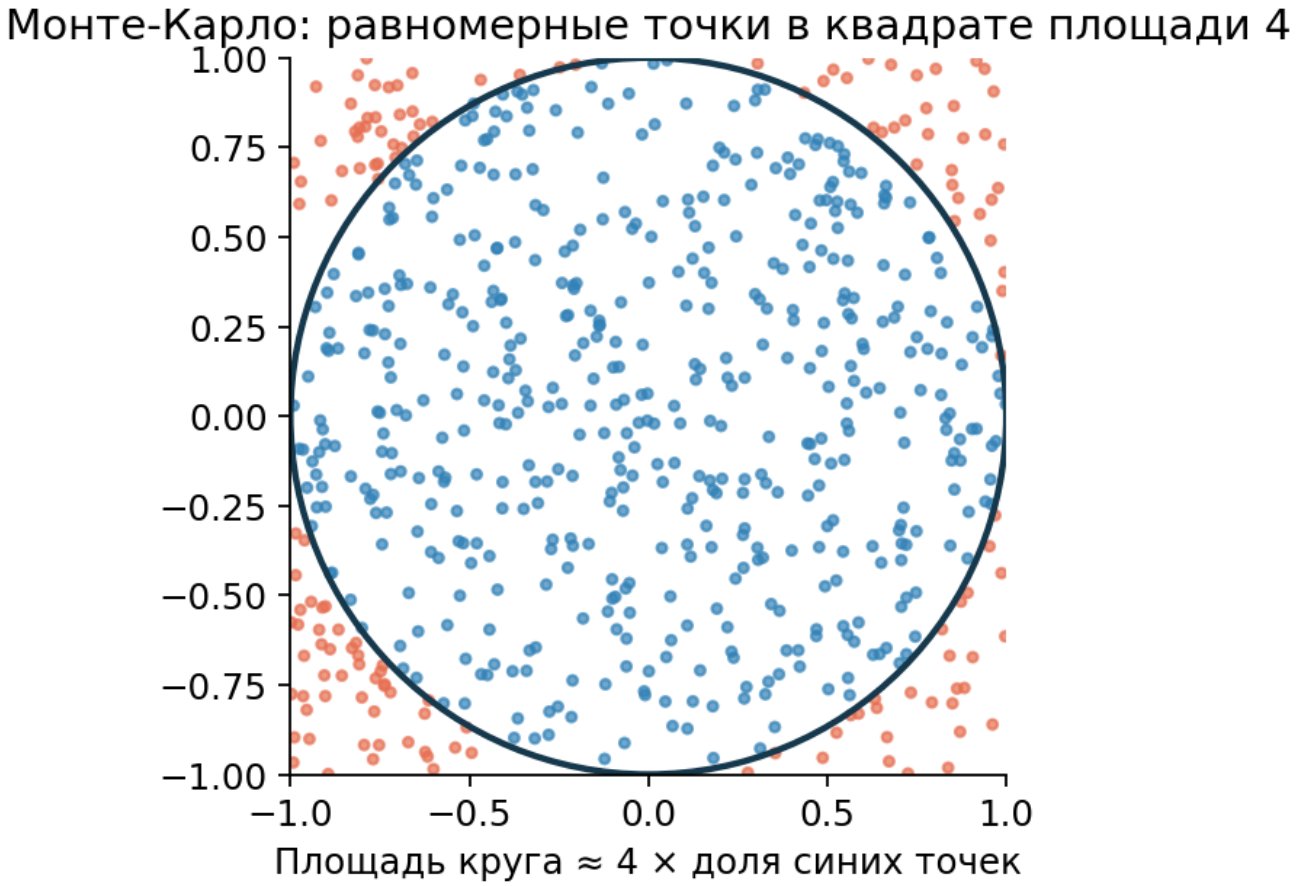
3.2. Как найти площадь фигуры и точность оценки?

Пусть фигура F лежит в прямоугольнике R площади A. Выбираем k независимых равномерных точек в R, $X_i=1$ при попадании в F, иначе 0. Тогда $p = P(X_i=1) = \text{area}(F)/A$ и оценка площади $\hat{A}=A\cdot(\Sigma X_i)/k$.

Почему так. $E[X_i]=p$, поэтому $E[\hat{A}]=Ap=\text{area}(F)$. $\text{Var}(\hat{A})=A^2p(1-p)/k \le A^2/(4k)$. Типичная абсолютная ошибка убывает как $1/\sqrt{k}$: для уменьшения ошибки в 10 раз требуется примерно в 100 раз больше точек.

Граница Хёффдинга: $P(|\hat{A}-\text{area}(F)| \ge A\varepsilon) \le 2\exp(-2k\varepsilon^2)$. Достаточно $k \ge \ln(2/\delta)/(2\varepsilon^2)$, чтобы абсолютная ошибка была не больше $A\varepsilon$ с вероятностью $\ge 1-\delta$. Это не гарантия малой относительной ошибки для очень маленькой фигуры.

Пример. Круг $x^2+y^2\le 1$ в квадрате $[-1,1]^2$, $A=4$. Из 10000 точек попало 7850: $\hat{A}=4\cdot 0{,}785=3{,}14$. Рисуем квадрат, вписанный круг, точки внутри и снаружи; каждый цвет соответствует $X=1$ или $X=0$. Для $\varepsilon=0{,}01$ и $\delta=0{,}05$ достаточно 18445 точек по консервативной границе выше. Выборка только на диагонали квадрата неверна: распределение не равномерно по площади.



Монте-Карло: равномерная выборка и проверка принадлежности

3.3. Как оценить размер дерева случайным путём?

Ответ на экзамене. Выбираем по одному ребёнку равномерно на каждом уровне до листа. Пусть вдоль пути числа детей $d_0, d_1, \ldots$. Оценка общего числа вершин, включая корень: $Z = 1 + d_0 + d_0 d_1 + \ldots$ до достигнутой глубины. Усредняем независимые Z.

Почему несмещённо. Вероятность посетить конкретную вершину v равна произведению $1/d$ вдоль пути к ней. Её вклад при посещении — обратная вероятность. Ожидание вклада каждой вершины равно 1; сумма ожиданий — число всех вершин. Это частный случай взвешивания по обратной вероятности включения.

Пример. У корня 2 ребёнка: левый — лист, у правого 3 листа. В дереве 6 вершин. При выборе левого получим $Z = 1 + 2 = 3$, правого — $Z = 1 + 2 + 2 \cdot 3 = 9$. Среднее $0{,}5 \cdot 3 + 0{,}5 \cdot 9 = 6$. Сильная неравномерность дерева может дать большую дисперсию; один путь не обеспечивает точности. Для неравномерного выбора нужно использовать реальные обратные вероятности, а не просто числа детей.

3.4. Как вывести число запусков для ошибки 1/e и 1/n?

Если вероятность успеха одного запуска $\geq p$ и запуски независимы, вероятность всех неудач $\leq (1-p)^k \leq \exp(-pk)$. Значит достаточно $k \geq \ln(1/\delta)/p$. Точная достаточная оценка при $0<p<1$: $k \geq \ln(\delta)/\ln(1-p)$, округлённая вверх.

При $\delta = 1/e$ достаточно $k \geq 1/p$. При $\delta = 1/n$ достаточно $k \geq \ln(n)/p$. Для Каргера $p \geq 2/[n(n-1)]$, значит соответственно порядка n^2 и $n^2 \log n$ запусков. Это число запусков, а не полное время: его нужно умножить на стоимость одного запуска. Повторения с одним и тем же состоянием

генератора случайности не являются независимыми.

4. Жадные алгоритмы

Статус: МОД встречается в вопросах про TSP; самостоятельные задачи о расписании, Хаффмане и доказательство жадности — дополнение по программе.

4.1. Как построить минимальное остовное дерево?

Ответ на экзамене. МОД — остовное дерево связного неориентированного взвешенного графа минимального суммарного веса. Краскал сортирует рёбра по весу и добавляет очередное, если оно соединяет разные компоненты; DSU проверяет это условие. Прим растит одно дерево, каждый раз выбирая самое лёгкое ребро наружу.

Доказательство свойства разреза. Пусть разрез не пересекается уже выбранными рёбрами и e — самое лёгкое ребро через него. Если оптимальный остов T не содержит e, добавим e: возникнет цикл. В этом цикле есть другое пересекающее разрез ребро f, причём $w(e) \le w(f)$. Замена f на e сохраняет остов и не увеличивает вес. Значит безопасный выбор можно включить в некоторый оптимум. Равные веса не мешают корректности.

Краскал: $O(m \log m)$, DSU добавляет $O(m\,\alpha(n))$, память $O(n)$ сверх списка рёбер. Прим с двоичной кучей $O((n+m)\log n)$, с матрицей $O(n^2)$; для плотного графа второй вариант удобен. Несвязный граф даёт минимальный остовный лес. Отрицательные веса разрешены.

Пример. Рёбра AB=1, BC=2, AC=3, CD=3, BD=4. Краскал берёт AB, BC; AC пропускает как цикл; CD берёт. Вес МОД равен 6.

4.2. Как решить задачу об оптимальном расписании?

Сначала уточнить постановку. Для максимального числа непересекающихся занятий на одном ресурсе каждое занятие — полуинтервал [s,f), все занятия одинаково ценны. Выбираем занятие с самым ранним временем окончания, затем первое совместимое с выбранным, и повторяем.

Обменный аргумент. В оптимальном расписании первое занятие o заканчивается не раньше самого раннего g. Заменим o на g: оставшиеся занятия по-прежнему помещаются, число не уменьшилось. После g остаётся такая же задача на более поздних интервалах. Индукция доказывает оптимальность. Сортировка $O(k \log k)$, проход $O(k)$.

Пример. [0,3), [1,2), [2,4), [3,5), [4,6). Берём [1,2), [2,4), [4,6): три занятия. Выбор по самому раннему началу мог бы взять [0,3) и потерять возможность получить три.

Для взвешенных занятий эта жадность неверна: [0,2) с ценностью 1 против [0,3) с ценностью 100. Нужна другая модель, обычно ДП $OPT(j)=\max(OPT(j-1), value(j)+OPT(p(j)))$. Если под «расписанием» в лекциях понимается минимизация опозданий, это иная задача: без конкретной постановки единого жадного алгоритма нет.

4.3. Как работает код Хаффмана и почему он оптимален?

Префиксный двоичный код: ни одно кодовое слово не является префиксом другого. Символы — листья двоичного дерева, код — путь из 0 и 1 от корня. Минимизируем Σ frequency(a)·depth(a).

Алгоритм. Два узла с минимальными частотами объединяем в родителя с суммарной частотой, возвращаем его в min-heap. Продолжаем до корня. Для k символов время $O(k \log k)$,

дерево O(k); выдача всех кодовых строк дополнительно стоит их суммарной длины.

Почему жадность верна. В некотором оптимальном полном двоичном дереве два наименее частых символа можно разместить в самых глубоких соседних листьях: перестановка менее частого вниз не увеличивает стоимость. Сжимаем эту пару в один символ суммы частот. Стоимость уменьшается на сумму частот пары, одинаково для всех соответствующих деревьев. Поэтому оптимальность меньшей задачи переносится на исходную.

Пример. Частоты A=5, B=2, C=1, D=1. Объединяем C,D→2; затем B и CD→4; затем A и BCD→9. Коды: A=0, B=10, C=110, D=111; стоимость 5·1+2·2+1·3+1·3=15 бит на 9 символов. Проверка: ни один код не префикс другого. Для единственного символа отдельно договориться о коде, например 0, и способе передачи длины сообщения.

5. Раскраска графа в три цвета

Статус: одна из центральных тем таблицы. Все варианты, применение, формула SAT, вывод вероятности, перебор малых множеств, K_5 и неясная «пентаграмма» разобраны здесь.

5.1. Постановка, применение и полный перебор

Ответ на экзамене. Требуется функция color:V→{1,2,3}, такая что для каждого ребра uv цвета концов различны. Проверяется допустимость, а не минимизируется число цветов. Для фиксированных трёх цветов задача NP-полна на произвольных графах. Пустой граф допускает раскраску; петля запрещает любую правильную раскраску, поэтому дальше рассматриваем граф без петель.

Полный перебор выбирает один из трёх цветов каждой вершине: 3^n назначений. Проверка полного назначения по рёбрам O(n+m), общее время O(3^n(n+m)); backtracking обнаруживает конфликты раньше. Память O(n+m) с хранением графа и текущих цветов. Фиксация цвета одной вершины снимает часть симметрии, но не меняет экспоненциальную базу.

Применение. В конфликтном графе вершина — экзамен, ребро — общие студенты; три цвета — три временных слота. Раскраска моделирует также распределение совместимых частот или ресурсов. Дополнительные ограничения по аудиториям в эту простую модель автоматически не входят.

5.2. Откуда алгоритм O*(2^n)?

Ответ на экзамене. Перебираю множество S вершин первого цвета. Проверяю, что S независимо, то есть внутри него нет рёбер. Остаток G[V\S] должен быть двудольным: раскрашиваю его BFS/DFS во второй и третий цвета. Если обе проверки пройдены, получена 3-раскраска.

Доказательство. При существующей 3-раскраске её первый цвет образует одно из перечисляемых независимых множеств, а остаток имеет два цвета и двудолен. В обратную сторону независимое S и двудольный остаток непосредственно дают корректные три цвета. Каждое из 2^n множеств проверяется за O(n+m), итого O(2^n(n+m)). Проверять только двудольность остатка недостаточно: вершины S не должны конфликтовать между собой.

Это и есть один стандартный смысл «свести перебор к двум вариантам»: принадлежит ли вершина первому цвету. Он не означает, что произвольный граф можно просто раскрасить двумя цветами. Если на лекции показывали другую реализацию перебора, на экзамене нужно явно определить её состояние.

5.3. Почему достаточно множеств размера не больше n/3?

В любой 3-раскраске хотя бы один цветовой класс имеет размер ≤⌊n/3⌋ по принципу Дирихле. Обозначим его S. Перебираем **все** S размера 0,1,…,⌊n/3⌋, для каждого проверяем независимость и двудольность остатка. Цвета не именованы заранее: маленький класс всегда можно назвать первым.

```
for k in 0..floor(n/3):
    for S in combinations(V, k):
        if independent(S) and bipartite(G[V\S]):
            return S + две доли остатка
return «3-раскраски нет»
```

Сложность с выводом. Число кандидатов $B(n)=\Sigma(i=0..\lfloor n/3\rfloor)\ C(n,i)$. По энтропийной оценке $B(n)\le 2^{(H(1/3)n)}$, где $H(p)=-p\log_2 p-(1-p)\log_2(1-p)$. $H(1/3)\approx 0{,}9183$, поэтому $B(n)=O^*(1{,}8899^n)$. Проверки дают $O(B(n)(n+m))$, память $O(n+m)$. Нельзя заменить сумму на $n/3$ или $3/n$: это размер кандидата, не число кандидатов.

Почему независимость — это не «множества без пересечений». S и V\S по определению не пересекаются. Дополнительное условие — отсутствие рёбер внутри S; это совсем другая проверка.

5.4. Как прогнать перебор подмножеств на K_5?

K_5: каждые две вершины смежны. Любой цветовой класс содержит не более одной вершины, поэтому $\chi(K_5)=5$. Предположение о 3-раскраске даёт две вершины одного цвета по Дирихле; между ними есть ребро — противоречие.

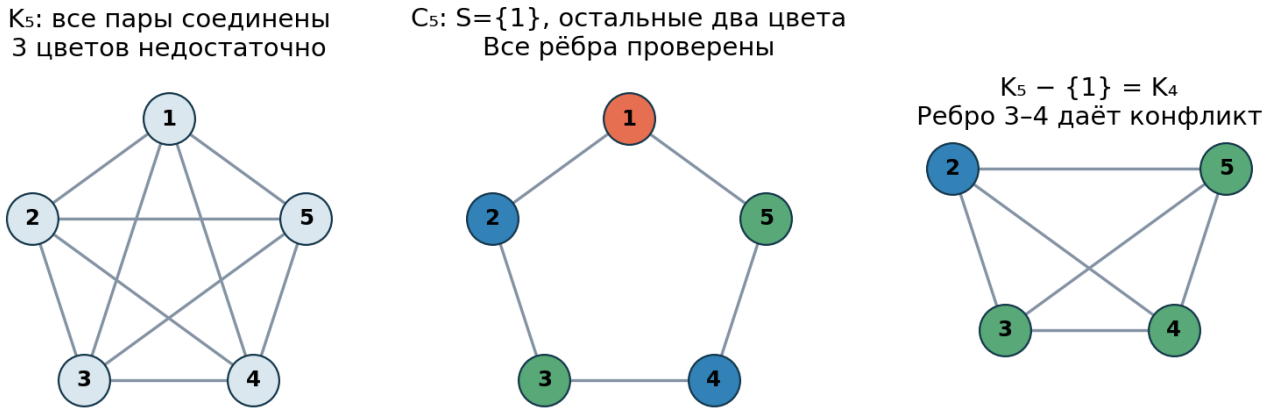
Для алгоритма $\lfloor 5/3\rfloor=1$. Кандидаты: ∅, {1}, {2}, {3}, {4}, {5}. Пустой S оставляет K_5, который содержит треугольник и не двудолен. Каждый одноэлементный S независим, но оставляет K_4, также содержащий треугольник. Все **шесть** кандидатов отвергнуты. Если пустой случай заранее отдельно исключён, остаются пять нетривиальных проверок; это соглашение, а не общая формула числа попыток.

Что показать BFS. В остатке K_4 красим вершину 2 синим; соседей 3,4,5 — зелёным. Ребро 3–4 имеет два зелёных конца: конфликт. Вывод алгоритма — доказанный отрицательный ответ, а не «метод сломался».

5.5. А если нарисовали пятиконечную звезду или C_5?

В исходном отзыве слово «пентаграмма» не задаёт однозначный граф: пересечения линий могут быть или не быть вершинами. Сначала уточнить вершины и рёбра. Нельзя подменять такой рисунок K_5.

Учебный пример C_5. Рёбра 12,23,34,45,51. Сам цикл нечётный и не двудолен. Берём S={1}; остаток — путь 2–3–4–5, двудольный. Цвета: 1 красный, 2 и 4 синие, 3 и 5 зелёные. Проверены все пять рёбер; получена 3-раскраска с первой же одноэлементной попытки. Для других рисунков надо выполнить такие же проверки, а не заучивать «пять попыток».



K_5, C_5 и конфликт двухцветной раскраски остатка

5.6. Как записать 3-раскраску булевой формулой?

Вводим x[v,c]: «вершина v имеет цвет c», c∈{R,G,B}. Для каждой вершины добавляем (x[v,R] ∨ x[v,G] ∨ x[v,B]) и три запрета пар: (¬x[v,R] ∨ ¬x[v,G]), (¬x[v,R] ∨ ¬x[v,B]), (¬x[v,G] ∨ ¬x[v,B]). Для каждого ребра uv и каждого цвета c добавляем (¬x[u,c] ∨ ¬x[v,c]).

Почему эквивалентно. Первое условие требует хотя бы один цвет, три следующих — не более одного. Рёберные условия исключают равенство цветов смежных вершин. Решение формулы однозначно задаёт правильную раскраску и наоборот. Это CNF с клаузами длины не более трёх; при определении 3-CNF как «ровно три литерала» короткие клаузы можно дополнить повторениями литералов. Переменных 3n, клауз 4n+3m.

Пример одного ребра a–b. Кроме четырёх вершинных условий для a и для b, выписываем (¬a_R∨¬b_R), (¬a_G∨¬b_G), (¬a_B∨¬b_B). Формула не является 2-SAT из-за трёхлитерального требования хотя бы одного цвета.

5.7. Вероятностный алгоритм: как получить 2-SAT и вероятность успеха?

Ответ на экзамене. Независимо для каждой вершины равновероятно запрещаю один из трёх цветов. Остаются два допустимых цвета — один булев выбор на вершину. Для каждого ребра запрещаю совпадение на каждом общем допустимом цвете. Это 2-SAT; решаю его за линейное от размера формулы время. Найденный ответ проверяю на исходном графе; при неудаче повторяю.

Явный пример. У a список {R,G}, переменная a=true означает R. У b список {G,B}, b=true означает G. Их единственный общий цвет G запрещает сочетание a=false,b=true. Клауза (a ∨ ¬b). Если оба списка {R,G}, ребро даёт (¬a∨¬b) и (a∨b), то есть различные значения. Не надо пытаться удалить произвольный литерал из каждой клаузы исходного SAT: так нельзя сохранять эквивалентность.

Вероятность. Фиксируем одну существующую правильную раскраску c*. У каждой вершины вероятность не запретить её цвет равна 2/3. Все n нужных цветов переживут ограничения с вероятностью (2/3)^n. При этом событии 2-SAT обязательно выполним. Значит вероятность успеха запуска p≥(2/3)^n. При k≥(3/2)^n ln(1/δ) вероятность пропустить существующее решение ≤δ. Время O(k(n+m)), или O*((3/2)^n) для постоянной ошибки; память O(n+m).

Односторонняя ошибка. Выданная раскраска всегда проверяема и правильна. Если раскраски нет, корректный решатель 2-SAT никогда её не выдаст. Но конечное число неудачных запусков на раскрашиваемом графе возможно: оно не даёт детерминированного доказательства отсутствия раскраски.

5.8. Почему 2-SAT решается за O(n+m)? Это двудольность?

Для каждой клаузы (a∨b) строим импликации ¬a→b и ¬b→a. На 2n литералах будет O(n+m) дуг для формулы выше. Находим компоненты сильной связности (SCC) алгоритмом Косарайю или Тарьяна за O(V+E). Формула невыполнима тогда и только тогда, когда для какой-либо переменной x литералы x и ¬x в одной SCC. Если противоречия нет, назначаем значения по топологическому порядку компонент, обрабатывая взаимно противоположные литералы согласованно.

Обоснование критерия. В одной SCC x и ¬x влекут друг друга, поэтому оба возможных значения приводят к противоречию. Если таких пар нет, конденсация — DAG, и выбор

значений в обратном порядке импликаций позволяет не назначать истинным литерал, ведущий к ложному; пара противоположных компонент получает противоположные значения.

Важное различие. Проверка двудольности решает обычную 2-раскраску. В вероятностном методе списки двух разрешённых цветов у соседей могут различаться; стандартное сведение идёт к 2-SAT и SCC, а не просто к проверке двудольности исходного графа. Фраза программы «двудольному графу 2SAT» требует такого уточнения.

6. Минимальные разрезы: Каргер и Каргер–Штейн

Статус: разрез, применение, стягивание на примере, вывод успеха, усиление до ошибок 1/e и 1/n, стоимость стягиваний и выбор n/√2 — из таблицы.

6.1. Что такое разрез и чем он отличается от потока?

Ответ на экзамене. Неориентированный разрез — разбиение V на непустые S и V\S; его рёбра имеют концы в разных частях. Вес разреза — сумма их весов, для невзвешенного графа — число рёбер. Глобальный минимальный разрез минимизирует этот вес по всем таким S. Минимальный s–t-разрез дополнительно требует $s \in S$, $t \notin S$.

Каргер ищет глобальный минимальный разрез в неориентированном графе. Теорема max-flow=min-cut связывает максимальный s–t-поток именно с минимальным s–t-разрезом в сети ёмкостей. Это разные постановки: нельзя просто назвать любой глобальный разрез двойственной задачей к одному произвольно выбранному потоку.

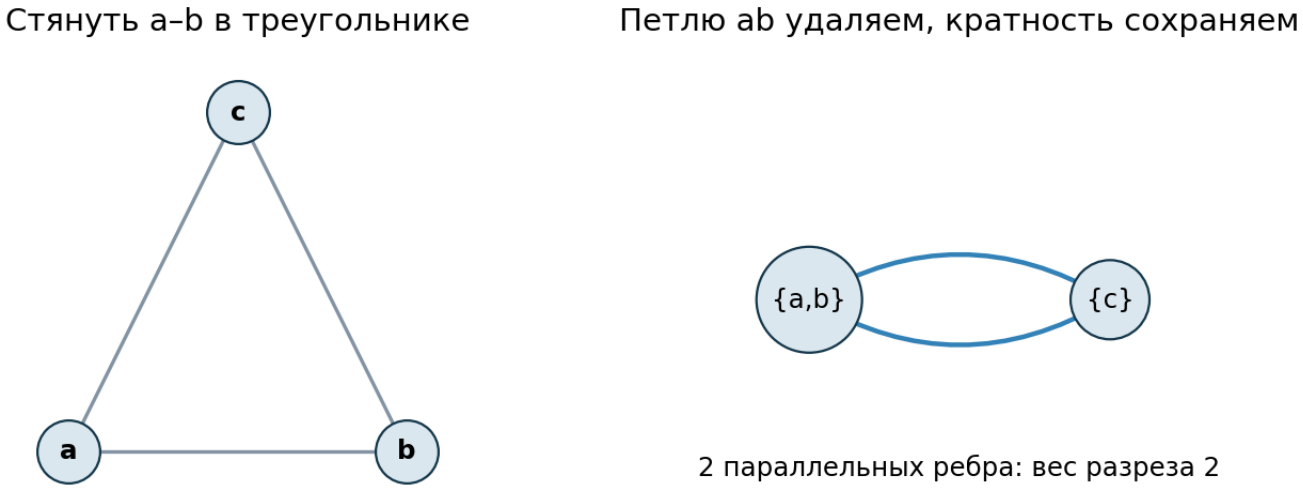
Применение. Разделение сети на слабо связанные группы; поиск минимальной суммарной пропускной способности, отключающей связь между двумя терминалами; сегментация через специально построенную s–t-сеть. Смысл веса и необходимость фиксированных терминалов зависят от модели. Если неориентированный граф несвязен, глобальный min-cut равен нулю; этот случай обрабатываем до анализа Каргера.

6.2. Как стягивать ребро и запускать Каргера?

Выбираем равномерное случайное ребро с учётом кратности и склеиваем его концы в супервершину. Петли удаляем, параллельные рёбра сохраняем. Повторяем до двух супервершин. Рёбра между ними образуют разрез исходного графа; храним состав супервершин, чтобы восстановить части.

Пример. Треугольник a,b,c. Стягиваем ab→X={a,b}. Рёбра ac и bc превращаются в два параллельных X–c. Это разрез {a,b}|{c} веса 2. Если заменить два параллельных ребра одним без веса, ответ станет неверным и распределение следующих стягиваний исказится.

Для весов $w \geq 0$ можно выбирать ребро с вероятностью пропорционально весу и суммировать веса параллельных рёбер. Отрицательные веса не входят в стандартный анализ. Стоимость одного стягивания зависит от структуры данных; это не автоматически O(1).



Стягивание ребра: две параллельные связи сохраняются

6.3. Откуда вероятность 2/[n(n−1)]?

Фиксируем один минимальный разрез C веса $\lambda>0$ в невзвешенном мультиграфе. Если ни одно его ребро ещё не стянуто, он сохранился. При r супервершинах каждый разрез супервершины имеет размер хотя бы λ, следовательно каждая степень $\geq\lambda$, а всего рёбер $\geq r\lambda/2$.

Вероятность следующим шагом выбрать ребро C не больше $\lambda/(r\lambda/2)=2/r$. Вероятность избежать его $\geq(r-2)/r$. Условные вероятности перемножаются:

```
P(сохранить C до двух вершин)
  >= product(r=3..n) (r-2)/r
   = (1·2·...·(n-2)) / (3·4·...·n)
   = 2 / (n(n-1)).
```

Если C сохранился, оставшиеся две супервершины задают этот минимальный разрез. Это нижняя граница вероятности найти хотя бы один минимум; при нескольких минимумах реальный успех может быть выше. Условие положительного λ существенно для этого вывода; несвязность уже обработана.

6.4. Сколько повторять Каргера и какова полная сложность?

Из §3.4 при $p\geq 2/[n(n-1)]$ достаточно $k=\lceil n(n-1)\ln(1/\delta)/2\rceil$ независимых запусков. Выбираем самый лёгкий найденный разрез. Для ошибки $\leq 1/n$ получается $O(n^2\log n)$ запусков, для $\leq 1/e$ — $O(n^2)$.

Не путать запуск и итерацию стягивания. Один запуск содержит $n-2$ стягивания. Условная простая реализация через матрицу весов активных супервершин: хранить взвешенные степени, выбирать вершину пропорционально степени и соседа пропорционально весу; это даёт равномерное ребро с учётом кратности. Линейный проход по строке для выборки и слияния стоит $O(n)$ на стягивание, запуск $O(n^2)$, память $O(n^2)$. С такой реализацией полное время усиленного алгоритма $O(n^4\log n)$. Копирование исходной матрицы $O(n^2)$ входит в запуск.

Другие реализации дают другие оценки. Если каждый раз заново просматривать весь список рёбер, не надо обещать $O(n^2)$ на плотном графе без анализа. На экзамене сначала назвать представление, затем посчитать затраты.

6.5. Что меняет Каргер–Штейн и почему n/√2?

Ответ на экзамене. При большом n независимо стягиваем две копии графа до $t=\lceil 1+n/\sqrt{2}\rceil$ вершин; рекурсивно решаем обе и выбираем лучший результат. Для малого n, например $n\le 6$, точно перебираем разрезы. Вариант с округлением t иначе возможен; формулы надо согласовать.

Вероятность сохранения фиксированного минимума до t вершин $\ge t(t-1)/[n(n-1)] \approx (t/n)^2 \approx 1/2$. Два независимых продолжения компенсируют этот риск. Если q(t) — вероятность рекурсивного успеха при сохранённом минимуме, то $q(n)\ge 1-(1-s\cdot q(t))^2$, где $s\ge 1/2$ при выбранном t. Для $s=1/2$ правая часть $q(t)-q(t)^2/4$; по уровням глубины $O(\log n)$ это даёт $q(n)=\Omega(1/\log n)$, а не автоматически константу.

Связь с временем. При $O(n^2)$ на создание и стягивание копий $T(n)=2T(n/\sqrt{2})+O(n^2)=O(n^2\log n)$. На каждом уровне суммарные квадраты размеров примерно n^2, уровней $O(\log n)$. Одно рекурсивное дерево — один вероятностный запуск. $O(\log^2 n)$ независимых таких запусков уменьшают ошибку до $1/n$; стандартная оценка полного времени $O(n^2\log^3 n)$. Для постоянной малой ошибки достаточно $O(\log n)$ запусков и $O(n^2\log^2 n)$ времени.

Смысл порога. Коэффициент $1/\sqrt{2}$ согласует вероятность выживания около 1/2 с двумя рекурсивными ветвями и квадратичной работой на уровень: $2\cdot(1/\sqrt{2})^2=1$. Это не произвольная «магическая» константа и не гарантия успеха 1/2 всего алгоритма.

7. Задача о коммивояжёре (TSP)

Статус: все основные методы, условия допустимого тура, нижняя граница МОД, локальный поиск, отжиг, ДП и доказательство 2-приближения — из таблицы.

7.1. Что перебираем и что значит допустимый тур?

Ответ на экзамене. Нужно найти замкнутый маршрут минимального веса, посещающий каждую вершину ровно один раз до возвращения в начало. Каждые две последовательные вершины, включая последнюю и первую, должны соединяться допустимым ребром. В неполном графе гамильтонова цикла может не быть.

Фиксируем начало s, перебираем (n−1)! перестановок остальных вершин, проверяем все рёбра тура и считаем сумму, в том числе возвращение в s. Для неориентированного графа обратный обход того же цикла эквивалентен, но деление на 2 не меняет Θ-класс. Прямая реализация O(n·(n−1)!), память O(n) сверх графа. Инкрементальный подсчёт частичного веса уменьшает затраты, но сам факториальный перебор не исчезает.

Ловушка. Перестановка вершин без проверки замыкающего ребра — ещё не тур. Посещение всех вершин в произвольном замкнутом обходе с повторами — ещё не гамильтонов цикл. Для обычной TSP предполагают n≥3; меньшие случаи зависят от соглашения.

7.2. Как отсекать по текущему пути и по МОД?

Пусть частичный путь начинается в s, заканчивается в v, посетил некоторое множество; U — непосещённые вершины, g — его вес, B — вес лучшего полного тура.

По текущему весу. При неотрицательных весах любое продолжение стоит ≥g. Если g≥B, улучшение невозможно. При отрицательных весах такая отсечка некорректна: будущие рёбра могут уменьшить суммарную стоимость.

По МОД. Для U≠∅ в неориентированной задаче используем

```
LB = g + MST(U) + min(u in U) w(v,u) + min(u in U) w(s,u).
```

В полном продолжении рёбра между непосещёнными вершинами образуют путь, покрывающий U; его вес ≥MST(U). Ещё нужны вход в U из v и выход из U к s; их веса не меньше соответствующих минимумов. При |U|=1 MST(U)=0, два подключения точно задают остаток. При U=∅ вместо формулы используем g+w(v,s), проверив наличие ребра.

Можно брать один минимум для обоих концов даже если они достигаются на одной вершине: граница остаётся нижней, хотя может стать слабее. Отсутствующее обязательное соединение или несвязность U делают ветвь невозможной. Эта формула не переносится без изменения на ориентированный TSP.

Числовой пример. g=7, U={a,b,c}, w(ab)=2, w(bc)=3, w(ac)=6; MST(U)=5. Минимальный вход из v равен 4, выход к s — 3. LB=19. При уже найденном B=18 ветвь отсекаем. Если граница меньше B, это не гарантирует хорошего тура: ветвь только остаётся на рассмотрении.

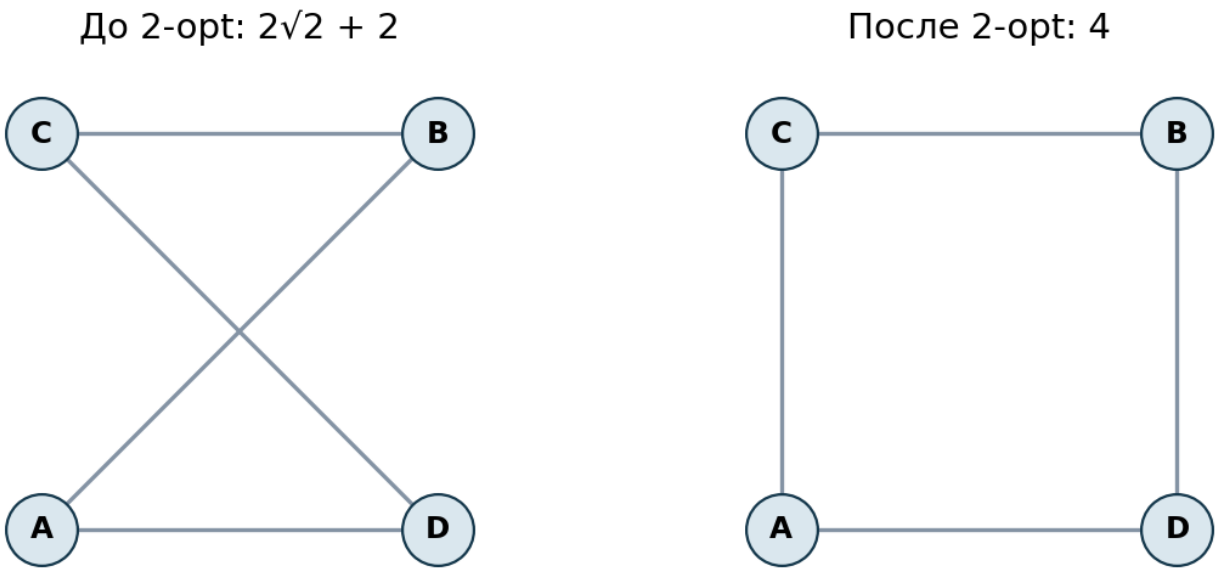
7.3. Что такое 2-окрестность и 2-opt?

Ответ на экзамене. В симметричном TSP ход 2-opt удаляет два несмежных ребра (a,b),(c,d) тура, добавляет (a,c),(b,d) и разворачивает участок от b до c. Это снова один цикл. Термин «2-окрестность» может также означать перестановку двух вершин: надо уточнить, какой ход выбран.

Изменение веса при 2-opt: $\Delta = w(a,c)+w(b,d)-w(a,b)-w(c,d)$. При $\Delta<0$ улучшение. В ориентированном графе разворот меняет веса всех внутренних дуг, поэтому этой формулы недостаточно.

Пример. Квадрат с точками A=(0,0), B=(1,1), C=(0,1), D=(1,0). Тур A–B–C–D–A имеет длину $2\sqrt{2}+2$. Меняем пересекающиеся AB,CD на AC,BD и разворачиваем B–C. Получается A–C–B–D–A длины 4. Выигрыш $2\sqrt{2}-2$.

Есть $\Theta(n^2)$ пар рёбер. Проверить Δ по матрице весов можно за $O(1)$, поэтому полный просмотр соседей $O(n^2)$. Применение выбранного разворота в обычном массиве занимает $O(n)$. За I проходов с одним улучшением на проход получаем $O(I\,n^2)$ плюс возможный финальный просмотр. Число проходов до локального оптимума не обязано быть полиномиальным. Локальный оптимум не гарантирует глобальный.



Устранение пересечения рёбер ходом 2-opt

7.4. Как работает имитация отжига? Почему не просто $O(n^2)$?

Выбираем случайного соседа. Улучшение $\Delta \le 0$ принимаем; ухудшение $\Delta > 0$ принимаем с вероятностью $\exp(-\Delta/T)$, где $T>0$ — температура. Постепенно понижаем T, сохраняя лучший найденный тур отдельно от текущего. Высокая T помогает выходить из локальных минимумов, низкая делает поведение более жадным.

Пример. При $\Delta=5$ и $T=10$ вероятность принять ухудшение $e^{-0,5}\approx 0,607$; при $T=1$ она $e^{-5}\approx 0,0067$. Решение принимают сравнением $U\sim \mathrm{Uniform}[0,1)$ с этой вероятностью.

Сложность зависит от реализации и бюджета. При K пробах, $O(1)$ вычислении Δ и $O(n)$ развороте принятого участка в массиве время $O(K+A\,n)$, где $A \le K$ — число принятых ходов. Если каждый раз пересчитывать весь тур, будет $O(Kn)$. Если на каждом температурном уровне полностью просматривать $\Theta(n^2)$ соседей, появится соответствующий множитель. Без

числа итераций и стоимости хода ответ «$O(n^2)$» необоснован.

Практическое геометрическое охлаждение $T \leftarrow \alpha T$, $0<\alpha<1$, не даёт гарантии найти оптимум за полиномиальное время. Теоремы о предельной сходимости требуют специальных условий и очень медленного охлаждения; это не обещание для ограниченного экзаменационного или программного прогона.

7.5. Двукратное приближение: условия, шаги, доказательство

Условия. Полный неориентированный граф, неотрицательные веса, симметрия и неравенство треугольника $w(u,v) \le w(u,x)+w(x,v)$. Это метрический TSP. Для произвольных весов доказательство не работает.

Алгоритм. Построить MST. Удвоить его рёбра, получить связный эйлеров мультиграф. Пройти эйлеров цикл. При повторном посещении вершины пропускать её и идти прямо к следующей ещё не посещённой; в конце вернуться в начало. Эквивалентная реализация — порядок первого посещения в DFS остова.

Доказательство по цепочке. Удалим одно ребро оптимального тура OPT. Получится остовное дерево, значит $w(MST) \le w(OPT)$ при неотрицательных весах. Удвоенный остов имеет вес $2w(MST)$. Пропуск промежуточных вершин не увеличивает длину по неравенству треугольника. Следовательно $w(тур) \le 2w(MST) \le 2w(OPT)$. Все вершины посещены, и полнота графа разрешает каждое сокращающее ребро.

Пример. Для четырёх углов единичного квадрата остов из трёх сторон весит 3. Удвоенный обход весит 6; сокращённый тур по периметру — 4. OPT=4, фактический коэффициент 1, а гарантия — не более 2.

Сложность. После построения остова обход и сокращение $O(n)$. Для полного графа с матрицей весов Прим $O(n^2)$, значит весь алгоритм $O(n^2)$, дополнительная память $O(n)$, входная матрица $O(n^2)$. Если использовать Краскала по всем $\Theta(n^2)$ рёбрам, получится $O(n^2 \log n)$. Без выбранного алгоритма МОД единой численной оценки нет.

7.6. Как решить TSP динамическим программированием?

Held–Karp: $D[S,v]$ — минимальный вес пути из s в v, посещающего ровно множество S, где $s,v \in S$. База $D[\{s\},s]=0$, невозможные состояния $+\infty$. Для $v \neq s$:

```
D[S,v] = min(u in S\{v}) (D[S\{v},u] + w(u,v)).
answer = min(v != s) (D[V,v] + w(v,s)).
```

Почему верно. Перед последней вершиной v в оптимальном пути стоит некоторая u. Удалив v, получаем оптимальный путь меньшего состояния: иначе заменили бы его более дешёвым и улучшили весь путь. Все возможные u перечислены. Состояний $O(n2^n)$, переходов до n, время $O(n^2 2^n)$, память $O(n2^n)$; предков храним для восстановления тура. Это точный экспоненциальный алгоритм, не локальный поиск.

8. Кратчайшие пути: BFS, Дейкстра, A*, Theta*, ALT

Статус: прогон Дейкстры, смысл A*, Theta* и ALT — из таблицы; подробный анализ эвристик и перевзвешивания — по программе.

8.1. Чем различаются BFS и Дейкстра? Как выполнить прогон?

BFS использует очередь и находит минимальное число рёбер от источника в невзвешенном графе (или при одинаковом положительном весе всех рёбер). Посещает каждую вершину и ребро за O(n+m), память O(n). Дейкстра выбирает непосещённую вершину с минимальным текущим d и релаксирует её исходящие рёбра. Корректен при неотрицательных весах. С двоичной кучей и списками смежности O((n+m)log n), с матрицей $O(n^2)$. Отрицательные рёбра нарушают стандартное доказательство; для них нужны другие алгоритмы.

Инвариант Дейкстры. При извлечении вершины u с минимальной актуальной оценкой её d[u] уже равно истинному расстоянию. Если более короткий путь существовал бы, первая ещё не обработанная вершина этого пути имела бы оценку не больше длины пути до u благодаря неотрицательности оставшегося участка — противоречие выбору u.

Прогон. Неориентированные рёбра s–a=2, s–b=5, a–b=1, a–t=5, b–t=1. Начало: d(s)=0, прочие ∞. Извлечь s: a=2,b=5. Извлечь a: b=3,t=7. Извлечь b: t=4. Извлечь t: ответ 4, путь s–a–b–t. Устаревшую запись b=5 в куче игнорируем. Остановка при первом обнаружении t дала бы неверный ответ 7; останавливаемся при извлечении актуальной минимальной записи.

8.2. Зачем нужен A* и какие условия обеспечивают оптимальность?

Ответ на экзамене. A* направляет поиск к цели, извлекая вершину с минимальным f(v)=g(v)+h(v), где g — найденная стоимость пути от старта, h — оценка остатка до цели. При h=0 это Дейкстра. Хорошая нижняя оценка уменьшает число раскрытых состояний, но худший случай не устраняет.

Допустимость: 0≤h(v)≤dist(v,t), h(t)=0. Согласованность: для каждого ребра u→v, h(u)≤w(u,v)+h(v). Согласованность влечёт допустимость для вершин, имеющих путь к t. При согласованной h вершину можно окончательно закрывать после извлечения. При допустимой, но несогласованной h надо разрешать повторное открытие при улучшении g; иначе оптимальность не гарантирована. Цель проверяем при извлечении из очереди, а не при первом добавлении.

Короткое доказательство оптимальности. До нахождения оптимальной цели в очереди существует вершина на оптимальном пути с корректным g и f≤OPT. Поэтому цель с g>OPT не может быть выбрана раньше неё. Для реализации без повторных открытий корректность g в этом аргументе обеспечивается согласованностью.

8.3. Как A* связан с перевзвешиванием? Почему путь сохраняется?

Определим $w'(u,v)=w(u,v)+h(v)-h(u)$. Для пути P из s в t телескопическая сумма даёт $w'(P)=w(P)+h(t)-h(s)$. При фиксированных концах добавляется одна и та же константа, поэтому порядок путей по длине не меняется.

При согласованности h все $w'\geq 0$. Дейкстра на w' использует расстояния $g'(v)=g(v)+h(v)-h(s)$, то есть выбирает по тому же порядку, что A* по g+h. При идеальной эвристике $h(v)=dist(v,t)$ каждое ребро, продолжающее кратчайший путь к t, имеет $w'=0$: $h(u)=w(u,v)+h(v)$. При нескольких кратчайших путях нулевых рёбер может быть много.

Уточнение к программе. A* не требует заранее знать оптимальный путь и «обнулять» именно его рёбра. Нулевое перевзвешивание — объяснение идеального случая через потенциал, а не доступная по умолчанию операция. Восстанавливать стоимость в исходных весах нужно с учётом добавленной константы.

8.4. Как выбирать эвристику? Ограничения и сложность A*

Для сетки с четырьмя соседями и единичной ценой шага $h=|dx|+|dy|$ — манхэттенская нижняя оценка. Для восьми соседей с ценами 1 и $\sqrt{2}$ подходит octile: $\max(|dx|,|dy|)+(\sqrt{2}-1)\min(|dx|,|dy|)$. Евклидово расстояние допустимо, если стоимость любого движения не меньше его геометрической длины. При дешёвых диагоналях манхэттенская оценка может завышать. Если минимальная цена единицы длины c, геометрическую оценку умножают на c.

Препятствия обычно увеличивают истинное расстояние, поэтому нижняя оценка без препятствий остаётся допустимой, но может быть слабой: в лабиринте A* много исследует в сторону тупика. Вычисление сложной h тоже стоит времени. В конечном явно заданном графе с согласованной h и двоичной кучей время $O((n+m)\log n)$ плюс вычисления h, память $O(n+m)$ с графом. В неявном пространстве состояний оценку часто выражают через фактор ветвления b и глубину d: число состояний может быть $O(b^d)$. Для несогласованной h с переоткрытиями прежняя оценка по одному раскрытию вершины не применяется.

8.5. Зачем Theta*, если координаты уже евклидовы?

Ответ на экзамене. Евклидовы координаты не разрешают произвольный прямой переход, если граф содержит только 4 или 8 соседей клетки. A* оптимален в этом дискретном графе, но его путь может быть ломаным и длиннее кратчайшего пути свободного движения. Theta* при релаксации проверяет видимость: можно ли соединить соседа v напрямую с родителем текущей вершины u, минуя u.

Если от parent(u) до v прямая не пересекает препятствия, пробуем $g(parent(u))+Euclidean(parent(u),v)$, иначе обычное $g(u)+w(u,v)$. Удалённые связи дают пути с произвольными углами. Проверка line-of-sight имеет свою стоимость и должна учитывать размеры агента и правила касания углов препятствий.

Пример. Из (0,0) в (3,2) без препятствий четырёхсвязная сетка даёт длину 5, геометрический отрезок — $\sqrt{13}\approx 3{,}606$. Отличаются разрешённые переходы, а не формула координат. Basic Theta* обычно даёт короткие гладкие пути, но не гарантирует точный глобальный непрерывный оптимум во всех конфигурациях.

8.6. ALT: как ориентиры дают эвристику?

ALT = A* + Landmarks + Triangle inequality. Заранее выбираем k ориентиров L и считаем расстояния до них. Для неориентированного графа $h(v)=\max(\ell\in L)\ |d(\ell,t)-d(\ell,v)|$. По неравенству треугольника каждая разность по модулю $\le d(v,t)$, значит максимум допустим и согласован.

Пример. $d(\ell,t)=12$, $d(\ell,v)=5$ даёт нижнюю оценку 7. Второй ориентир с расстояниями 9 и 7 даёт 2, выбираем $\max(7,2)=7$. Это может быть значительно лучше евклидовой оценки вокруг больших препятствий.

Для ориентированного графа модуль неприменим в таком виде: используем max по доступным конечным величинам 0, $d(\ell,t)-d(\ell,v)$, $d(v,\ell)-d(t,\ell)$. Для этого нужны расстояния от ориентиров и до них (запуск на обратном графе). Предобработка k запусков Дейкстры для неориентированного графа, обычно 2k для ориентированного, хранение $O(kn)$, вычисление h за $O(k)$. Если веса меняются, корректность старых оценок надо проверять: удешевление рёбер может сделать их завышенными.

9. Потоки: Форд–Фалкерсон и Гольдберг (push–relabel)

Статус: назначение обратного ребра, краевые случаи, сложности, инварианты и доказательство максимальности — из таблицы.

9.1. Что такое поток, остаточная сеть и max-flow=min-cut?

Сеть — ориентированный граф с истоком s, стоком t≠s и ёмкостями c(u,v)≥0. В модели по исходным дугам 0≤f(e)≤c(e); у всех v≠s,t входящий поток равен выходящему. Величина потока — чистый выход из s. При наличии исходных дуг в обе стороны удобно хранить каждую со своей отдельной парой остаточных дуг, не смешивая их.

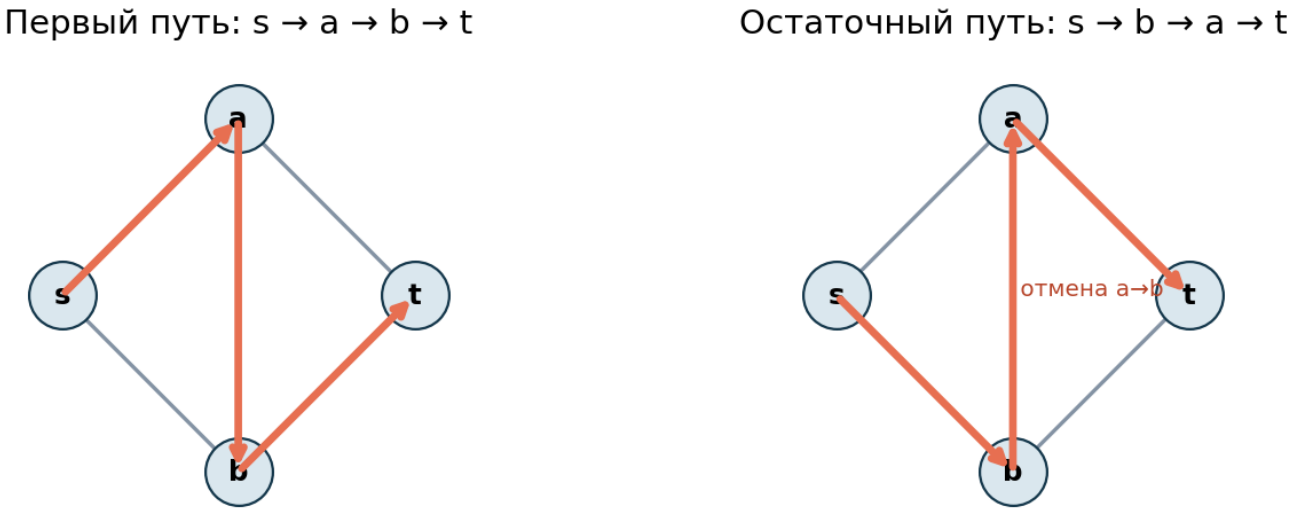
Остаточная ёмкость вперёд по исходной дуге e равна c(e)−f(e), назад — f(e). В остаточной сети присутствуют положительные остаточные ёмкости. Обратная дуга означает право отменить ранее принятое решение, а не физически отрицательную поставку по исходному ребру.

Для разреза S с s∈S,t∉S величина любого потока равна выходящему через разрез минус входящему и ≤Σ(u∈S,v∉S)c(u,v). Если остаточного пути s→t нет, берём S всех достижимых из s в остаточной сети. Все исходящие дуги разреза насыщены, входящий поток через него нулевой; тогда |f|=c(S,V\S). Это одновременно доказывает максимальность потока, минимальность данного s–t-разреза и теорему max-flow=min-cut.

9.2. Форд–Фалкерсон: шаги, обратные рёбра и пример

Начинаем с нулевого потока. Пока есть остаточный путь P от s к t, находим Δ=min residual(e) вдоль P и проталкиваем Δ: по прямым дугам добавляем поток, по обратным отменяем. Реконструируем новые остаточные ёмкости. Когда пути нет, используем доказательство §9.1.

Пример, где нужна отмена. Ёмкости всех дуг равны 1: s→a, s→b, a→b, a→t, b→t. Плохой первый путь s→a→b→t даёт поток 1. Второй остаточный путь s→b→a→t использует обратную к a→b дугу b→a. Отменяем a→b, добавляем s→b и a→t. Теперь идут два маршрута s→a→t и s→b→t, суммарный поток 2. Без обратной дуги алгоритм застрял бы на 1.



Второй остаточный путь отменяет неудачный выбор первой итерации

9.3. Сложность и краевые случаи Форда–Фалкерсона

При целых ёмкостях каждое увеличение повышает целый поток хотя бы на 1. Для поиска пути за O(n+m) общее время O((n+m)F), где F — величина максимального потока; в стандартном учёте достижимой сети часто пишут O(mF). Это псевдополиномиальная оценка: F может быть экспоненциальным от битовой длины ёмкостей. Память остаточной сети O(n+m).

При рациональных ёмкостях можно масштабировать до целых, но оценка зависит от знаменателей. При произвольных иррациональных ёмкостях неудачный выбор увеличивающих путей может привести к бесконечному процессу и даже неверному пределу. Вариант Эдмондса–Карпа выбирает кратчайший по числу рёбер остаточный путь BFS и работает $O(nm^2)$ независимо от численных значений ёмкостей в обычной модели арифметики.

Края. Нет s–t-пути — ответ 0. Нулевые ёмкости просто не дают остаточных дуг. Петли не улучшают поток s–t и могут быть исключены. Параллельные дуги надо учитывать корректно. s=t не является стандартной задачей максимального потока: сначала уточняем постановку. Вещественная машинная арифметика требует аккуратного сравнения с нулём; математическая гарантия точного алгоритма не снимает ошибок округления.

9.4. Что такое предпоток и высоты в алгоритме Гольдберга?

Ответ на экзамене. Push–relabel временно разрешает избыток в промежуточных вершинах и локально передвигает его по остаточной сети. Предпоток соблюдает ёмкости, но для $v \neq s$ допускает $e(v) = \text{вход} - \text{выход} \geq 0$. Активна вершина $v \neq s,t$ с $e(v) > 0$.

Высоты h целые, $h(s)=n$, $h(t)=0$. Инвариант допустимой разметки: для любой остаточной дуги $u \to v$ имеем $h(u) \leq h(v)+1$. Инициализация: все прочие высоты 0, насыщаем исходящие дуги из s. При стандартной форме сети можно не учитывать дуги в s и из t; общая парная остаточная реализация также сохраняет инварианты.

```
push(u,v):
    требуется e(u)>0, residual(u,v)>0, h(u)=h(v)+1
    delta = min(e(u), residual(u,v))
    увеличить поток вдоль остаточной дуги на delta
    e(u) -= delta; e(v) += delta

relabel(u):
    требуется u активна и нет допустимой для push дуги
    h(u) = 1 + min(h(v) по residual(u,v)>0)
```

Для активной вершины существует остаточный путь обратно к s: избыток возник из ранее протолкнутого потока и может по обратным дугам вернуться. Поэтому минимум в relabel определён. Высота при relabel строго возрастает, а допустимость разметки сохраняется. Push разрешён только «вниз на один», поэтому открываемая обратная остаточная дуга тоже не нарушает неравенство высот.

9.5. Как доказать, что в конце получится максимальный поток?

Шаг 1 — допустимость. Push сохраняет ограничения ёмкостей и не создаёт отрицательного избытка в промежуточных вершинах. После исчезновения всех активных вершин $e(v)=0$ для $v \neq s,t$: предпоток стал обычным потоком.

Шаг 2 — нет увеличивающего пути. При допустимой разметке любой остаточный путь от s к t длины k даёт $h(s) \leq h(t)+k=k$. Но простой путь имеет не более $n-1$ рёбер, а $h(s)=n$. Противоречие. Значит остаточного s–t-пути нет на протяжении работы, даже если текущий

объект ещё не поток.

Шаг 3 — максимальность. В конце есть допустимый поток и нет увеличивающего пути, следовательно по §9.1 поток максимален. Само отсутствие остаточного s–t-пути у промежуточного предпотока ещё не доказывает готовность ответа: лишний избыток должен быть устранён.

Пример. Сеть s→a ёмкости 3, a→t ёмкости 2, n=3. Насыщаем s→a: e(a)=3, h(a)=0. Поднимаем a до 1, проталкиваем 2 в t. Остаётся e(a)=1, но a→t насыщена. Поднимаем a до h(s)+1=4, возвращаем 1 по обратной a→s. В конце s→a=2, a→t=2, максимум 2.

9.6. Почему завершается и как сравнить сложности?

Высоты не убывают. У активной вершины есть простой остаточный путь до s длины $\le n-1$, поэтому $h(v) \le h(s)+(n-1)=2n-1$. Каждая вершина поднимается $O(n)$ раз, всего $O(n^2)$ relabel. Различают насыщающее проталкивание, которое обнуляет остаточную ёмкость дуги, и ненасыщающее, которое устраняет избыток в исходной вершине. Стандартный анализ числа таких операций даёт для общего push–relabel $O(n^2m)$ времени при организации просмотра текущих дуг; память $O(n+m)$.

Откуда оценка. Между двумя насыщениями одной и той же направленной остаточной дуги высоты концов должны вырасти: чтобы вновь открыть её через обратное проталкивание, требуется противоположный допустимый перепад. Поэтому насыщений $O(nm)$. Потенциал суммы высот активных вершин уменьшается при ненасыщающем push; relabel и насыщения увеличивают его суммарно не более $O(n^2m)$, откуда эта же граница числа остальных операций. Полные реализации используют указатель текущей дуги, чтобы не сканировать список заново без ограничения.

Вариант relabel-to-front имеет оценку $O(n^3)$; конкретные эвристики и способы выбора активных вершин дают иные оценки. Их нельзя без названия реализации смешивать. В отличие от $O(mF)$ у целочисленного Форда–Фалкерсона, эти оценки не зависят от величины максимального потока. Оба алгоритма полагаются на остаточные обратные дуги, но первый растит поток по целым путям, а второй двигает избыток локально.

10. Изоморфизм графов и алгоритм Ульмана

Статус: определения, назначение, перебор с матрицей и небольшой пример изоморфизма — из таблицы и программы.

10.1. Точный изоморфизм и поиск подграфа — одно и то же?

Изоморфизм G и H — биекция $\varphi: V(G) \to V(H)$, сохраняющая смежность в обе стороны: $uv \in E(G)$ тогда и только тогда, когда $\varphi(u)\varphi(v) \in E(H)$. Для взвешенных или размеченных графов отдельно требуем сохранения соответствующих меток. Должны совпадать n,m, мультимножества степеней и многие другие инварианты; их совпадение необходимо, но недостаточно.

Поиск подграфа — инъекция меньшего шаблона P в большой граф T. Для обычного, неиндуцированного подграфа сохраняем рёбра P; дополнительные рёбра между образами разрешены. Для индуцированного подграфа требуется сохранить и отсутствие рёбер. Это различие меняет проверки.

Пример изоморфизма. У первого пути рёбра ab,bc,cd; у второго 13,32,24. Биекция a→1,b→3,c→2,d→4 сохраняет все рёбра и нерёбра. **Контрпример достаточности степеней:** цикл C_6 и два непересекающихся треугольника имеют по 6 вершин степени 2, но первый связен, второй нет.

10.2. Как работает алгоритм Ульмана?

Ответ на экзамене. Ульман перебирает инъективное отображение вершин шаблона в целевой граф, храня матрицу кандидатов $M[i,j] \in \{0,1\}$. Единица означает, что вершину i шаблона ещё можно сопоставить вершине j целевого графа. Сначала фильтруем по меткам и степеням: для подграфа $\deg_P(i) \le \deg_T(j)$, для полного изоморфизма степени равны.

Уточнение матрицы. Если M[i,j]=1, то для каждого соседа x вершины i должен существовать сосед y вершины j с M[x,y]=1. Если такого y нет, обнуляем M[i,j]. Повторяем до неподвижной точки. Пустая строка означает невозможность данной ветви. Затем выбираем строку, фиксируем единственного кандидата, исключаем его столбец для остальных строк и рекурсивно уточняем матрицу. На полном назначении проверяем все нужные рёбра; для индуцированного варианта также нерёбра.

Почему корректно. Истинное отображение никогда не удаляется фильтром: каждому соседу шаблона соответствует настоящий сосед образа. Перебор рассматривает все оставшиеся инъекции. Фильтр — необходимое условие, но он может оставить несовместимые кандидаты; сам по себе не решает задачу.

Пример. Шаблон — путь x–y–z, цель — a–b–c. Для y остаётся только b по степени 2. Для x,z кандидаты a,c. Фиксируем x→a: столбец a исключён, значит z→c. Второе решение — симметричное отражение.

При p вершинах шаблона и q цели число возможных инъекций до $q!/(q-p)!$, поверх него затраты фильтрации и проверки. Для полного изоморфизма — до n! в этом переборном алгоритме, память матрицы O(pq) плюс состояние поиска. Это оценка Ульмана, не доказательство NP-полноты точного изоморфизма. Поиск произвольного подграфа NP-полон; для обычного изоморфизма статус иной, и называть его NP-полным как установленный факт нельзя.

Применение. Сопоставление молекулярных структур, поиск шаблонов в схемах, анализ структурных моделей и дедупликация графов с переименованными вершинами.

11. Симбиоз ИИ и A*

Статус: в таблице спрашивали, зачем ИИ нужен A*; три направления моделей ниже — дополнение по программе. Конкретные статьи и архитектуры в исходнике не названы, поэтому это общие корректные схемы, а не пересказ определённой лекции.

11.1. Какую проблему решает ИИ и где теряются гарантии?

Ответ на экзамене. A* требует хорошей оценки оставшейся стоимости. Модель может предсказывать её по карте, цели и динамике агентов, а также выбирать порядок раскрытия вершин. Это уменьшает практический поиск, но качество и гарантии надо проверять раздельно. Малое среднее отклонение модели не означает допустимость: единственное завышение может нарушить оптимальность.

Безопасный пример: держать f=g+h_safe с доказанной согласованной нижней оценкой, а нейросетевой прогноз использовать только для разрешения равенств f. Порядок между равными ключами меняется, доказательство оптимальности сохраняется. Простое min(h_model,h_safe) даёт допустимость при допустимой h_safe, но не обязательно согласованность; тогда могут требоваться переоткрытия. max(h_model,h_safe) без доказанной нижней границы для модели может завышать.

11.2. Как генетический алгоритм выбирает модели?

Особь кодирует архитектуру, параметры эвристики или конфигурацию обучения; популяцию оценивают на независимых задачах поиска. Fitness может учитывать время, число раскрытых вершин, память, долю найденных путей, стоимость пути и нарушения требований безопасности. Далее отбор, скрещивание, мутации, повторная оценка. Генетический поиск выбирает кандидатов, но сам по себе не доказывает их оптимальность и не устраняет переобучение. Нужны отдельные обучающая, валидационная и тестовая выборки карт.

Пример проверки. Модель A раскрывает 200 состояний и всегда выдаёт оптимум на тесте, B раскрывает 80, но даёт путь на 20% длиннее. Какая лучше — зависит от поставленной цели; выдавать B за точный A* без изменения гарантий нельзя.

11.3. RNN, динамические вероятности и многоагентное планирование

Рекуррентная сеть использует историю движений для прогноза будущей занятости клеток. Состояние поиска расширяют до (позиция, время), для нескольких агентов — ещё и согласованность совместных планов. Запрещают одновременное занятие одной клетки и встречный обмен через одно ребро; могут применяться таблицы резервирования или конфликтный поиск. Если планирование идёт только по предсказанию, ошибки прогноза требуют перепланирования.

Вероятность столкновения нельзя произвольно подставить вместо нижней оценки длины пути: нужно определить критерий, например ожидаемую стоимость или ограничение на риск. Изменение прогноза во время поиска меняет модель задачи; гарантии статического A* к ней автоматически не переходят.

11.4. CNN и трансформеры по изображениям препятствий

CNN выделяет локальную геометрию, трансформер может учитывать дальние связи на карте; выходом служат карта занятости, оценка проходимости, стоимости или подсказки для h. Планировщик затем ищет по явно построенному графу или пространству состояний. Необходимо учесть масштаб пикселей, размеры робота, неизвестные клетки и погрешность распознавания; ошибочно «свободный» пиксель может означать физическое столкновение.

Что сказать на уточнение «зачем ИИ». Чтобы использовать структуру среды и историю для уменьшения числа рассматриваемых состояний или адаптации плана. Для оптимальности в заданном графе нейросеть не обязательна. Сравниваем с BFS/Дейкстрой/A* на одинаковых картах, отдельно измеряя скорость, длину пути и число конфликтов.

12. Динамические графы и поддержка МОД

Статус: дополнение по программе; явных самостоятельных вопросов в записях не найдено.

12.1. Чем динамический граф отличается от статического?

В статическом графе структура не меняется между запросами. В динамическом поступают обновления: вставка/удаление ребра, изменение веса, иногда вершин. Инкрементальный режим — только вставки, декрементальный — только удаления, полностью динамический — оба вида. Цель — использовать предыдущий результат и отвечать быстрее, чем решать задачу заново после каждого изменения.

Примеры: изменяющиеся дороги и пробки, связи в сети, потоки дружбы/подписок, структура зависимостей. Изменение графа и движение агента по неизменному графу — разные понятия.

12.2. Как обновлять МОД после одного изменения?

Вставка ребра uv веса w. Если u,v в разных компонентах текущего остовного леса, добавляем ребро. Если в одной, возникает единственный цикл: найдём максимальное по весу ребро e на пути u→v в дереве. При w<w(e) заменим e на новое; иначе старый МОД остаётся минимальным. При равенстве можно выбрать любой из двух остовов.

Доказательство. Цикловое свойство разрешает исключить самое тяжёлое ребро возникшего цикла. Для более формального обмена сравниваем любой новый оптимум с добавленным ребром и устраняем возникший цикл без увеличения веса; старый остов уже был оптимален до вставки.

Удаление ребра. Если оно не в МОД, дерево остаётся оптимальным. Если в МОД — удаление разбивает его на A,B; находим самое дешёвое оставшееся ребро через этот разрез и соединяем им компоненты. Если такого нет, исходный граф стал несвязным и нужен лес.

Увеличение веса древесного ребра сводим к поиску лучшей замены через его разрез; уменьшение недревесного — к вставке/циклу. Уменьшение веса древесного или увеличение недревесного не требует замены для сохранения некоторого МОД.

Пример. Дерево AB=1, BC=4, CD=2. Вставляем AD=3: цикл A–B–C–D–A, самое тяжёлое BC=4, заменяем на AD=3; вес уменьшается с 7 до 6. Для простой реализации поиск максимума по дереву O(n), поиск замены при удалении просмотром всех рёбер O(n+m). Быстрые динамические деревья улучшают запросы на пути, но поиск замены при удалении требует дополнительной структуры; обещать O(log n) всем обновлениям только из-за наличия link-cut tree нельзя.

13. Редакционное расстояние: Левенштейн и Вагнер–Фишер

Статус: определение, применение, сложность, заполнение таблицы, место ответа и восстановление редакционного предписания — из таблицы.

13.1. Что именно измеряет расстояние Левенштейна?

Ответ на экзамене. Это минимальное число вставок одного символа, удалений одного символа и замен одного символа, превращающих строку A в B; все операции стоят 1. Совпадение символов стоит 0. Перестановка соседних символов не входит в обычное расстояние Левенштейна: это другая модель, например Дамерау–Левенштейн.

Применяется для исправления опечаток, поиска похожих строк и сравнения последовательностей. Для базовой версии $d(A,A)=0$, $d(A,B)=d(B,A)$, $d(A,C)\le d(A,B)+d(B,C)$, поскольку последовательности исправлений можно обращать и склеивать.

Редакционное предписание — конкретная последовательность допустимых операций, переводящая A в B. В отзыве написано «редукционное предписание»; в данном контексте ожидается именно редакционное. Расстояние — длина кратчайшего такого предписания, а не само предписание.

13.2. Как построить таблицу и вывести переход?

D[i,j] — расстояние между префиксами A[0:i] и B[0:j]. Базы: D[i,0]=i (всё удалить), D[0,j]=j (всё вставить). Для i,j≥1:

```
D[i,j] = min(
    D[i-1,j] + 1,                 # удалить A[i-1]
    D[i,j-1] + 1,                 # вставить B[j-1]
    D[i-1,j-1] + (A[i-1]!=B[j-1]) # совпадение или замена
)
```

Почему верно. У оптимального предписания на префиксах последняя операция относится к одному из трёх случаев. После её удаления остаётся оптимальное предписание меньших префиксов, иначе всё решение можно было бы улучшить. Все случаи перечислены. Заполняем по строкам или диагоналям так, чтобы зависимости уже были известны. Ответ в правом нижнем углу D[|A|,|B|], не минимум во всей таблице.

Пошаговый пример A=«кот», B=«кит».

A / B	ε	к	и	т
ε	0	1	2	3
к	1	0	1	2
о	2	1	1	2
т	3	2	2	1

Ячейка D[2,2]: min(1+1,1+1,0+1)=1, потому что «о»≠«и». Ячейка D[3,3]: min(2+1,2+1,1+0)=1, потому что «т»=«т». Расстояние 1.

13.3. Как восстановить ответ обратным ходом?

Начинаем в (|A|,|B|). Проверяем, какой переход действительно достигает текущего значения. Если D[i,j]=D[i−1,j−1]+cost, идём по диагонали: совпадение либо замена. Если D[i,j]=D[i−1,j]+1 — удаление; если равно D[i,j−1]+1 — вставка. У границы возможны только оставшиеся удаления или вставки. При равенстве нескольких вариантов оптимальных предписаний несколько; можно выбрать любое. Записанные при обратном ходе операции разворачиваем для исполнения слева направо, учитывая сдвиг индексов после вставок и удалений.

Пример. (3,3)→(2,2): совпадение «т». (2,2)→(1,1): замена «о» на «и». (1,1)→(0,0): совпадение «к». После разворота: оставить «к», заменить «о»→«и», оставить «т». Проверка применения даёт «кит» за одну платную операцию.

Время построения Θ(|A||B|), память полной таблицы Θ(|A||B|), обратный ход O(|A|+|B|) плюс вывод. Для одного значения достаточно двух строк и O(min(|A|,|B|)) памяти; простой обратный ход по уничтоженным строкам уже невозможен. Восстановление с линейной памятью требует отдельного алгоритма деления пополам, например подхода Хиршберга, а не просто «оставить две строки».

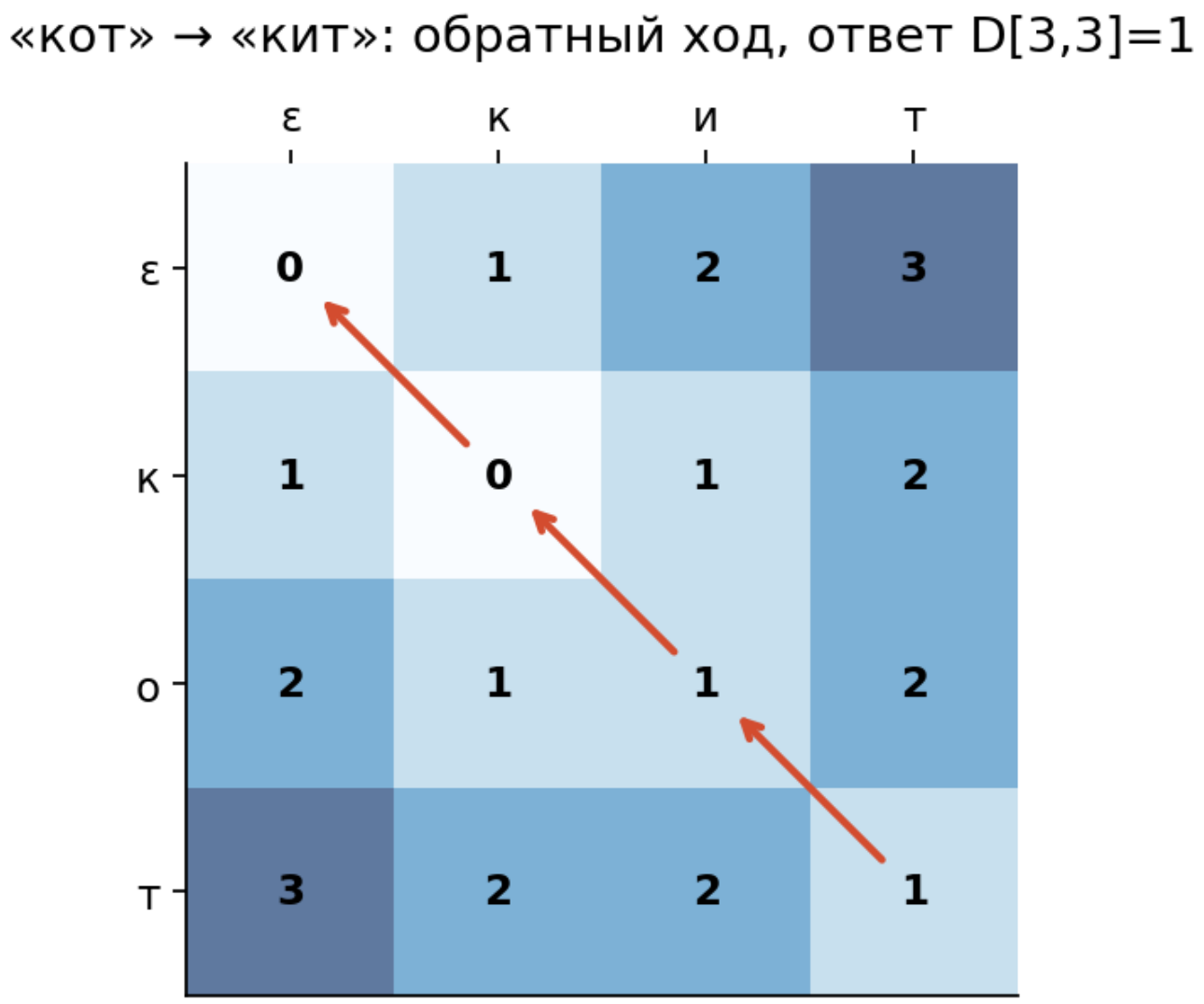


Таблица Левенштейна и обратный ход

13.4. Как свести задачу к кратчайшему пути?

Строим DAG с вершинами (i,j), 0≤i≤|A|,0≤j≤|B|. Дуги (i,j)→(i+1,j) веса 1 — удаление, →(i,j+1) веса 1 — вставка, →(i+1,j+1) веса 0 или 1 — совпадение/замена. Любой путь от (0,0) до (|A|,|B|) задаёт выравнивание и предписание. Его вес — число исправлений. Переходы ДП вычисляют кратчайшие пути в топологическом порядке. Визуально таблица — решётка, а восстановление — путь по её ячейкам.

Ловушка. Для поиска подстроки, а не преобразования всей строки, начальные условия и способ выбора конечной ячейки могут меняться. Не переносите такой вариант на обычную задачу Левенштейна без изменения постановки.

14. Префикс-функция и Кнут–Моррис–Пратт

Статус: определение, вычисление на примере, сравнение с наивным поиском, откаты и доказательство линейности, рисунок «( )x( )y» — из таблицы.

14.1. Что такое префикс-функция? Пример

Ответ на экзамене. π[i] — длина наибольшего собственного префикса строки S[0:i+1], одновременно являющегося её суффиксом. Собственный значит короче всей рассматриваемой строки. π[0]=0. Это длина, а не индекс найденного символа и не число появлений образца.

Пример S=«abababca».

i	0	1	2	3	4	5	6
S[i]	a	b	a	b	a	c	a
π[i]	0	0	1	2	3	0	1

При i=4 префикс «aba» совпадает с суффиксом «aba», длина 3. При i=5 пробуем продолжить границу длины 3: S[3]=b≠c; откат j=π[2]=1, S[1]=b≠c; откат j=π[0]=0, S[0]=a≠c. Получаем 0. При i=6 символ a начинает границу длины 1.

14.2. Наивное построение π и линейный алгоритм

Наивно для каждого i проверять все длины k=1..i и сравнивать префикс и суффикс посимвольно: O(M³) для строки длины M. Если для каждого i независимо искать границу через более организованные сравнения, возможен O(M²) вариант; поэтому слово «наивный» без описания реализации неоднозначно.

Линейный алгоритм использует уже вычисленные границы:

```
pi[0] = 0
for i in 1..M-1:
    j = pi[i-1]
    while j > 0 and S[i] != S[j]:
        j = pi[j-1]
    if S[i] == S[j]:
        j += 1
    pi[i] = j
```

При M=0 возвращаем пустой массив и не обращаемся к pi[0]. **Почему откат именно π[j−1].** Если граница длины j не продолжилась новым символом, следующая возможная граница — наибольшая граница уже совпавшего префикса длины j. Любая меньшая подходящая граница находится в этой цепочке. Откат на j−1 мог бы перебирать ненужные длины, а откат сразу в 0 пропускал бы допустимые границы.

14.3. Почему вложенный while не делает алгоритм квадратичным?

Ответ на экзамене. Внешний цикл $M-1$ раз сдвигает i вправо. Переменная текущей длины границы j увеличивается не более чем на 1 за шаг, то есть суммарно $O(M)$. Каждый проход while строго уменьшает j. Так как $j \geq 0$, суммарное число уменьшений тоже $O(M)$. Между итерациями pi[i] равно полученному j, поэтому присваивание j=pi[i−1] не создаёт дополнительных «бесплатных» повышений. Итого $O(M)$ времени, $O(M)$ памяти для π.

Это амортизированный анализ одной последовательности сравнений, не вероятностная оценка. Худшее число откатов в одном шаге может быть $\Theta(M)$, но такое поведение не может повторяться M раз без новых увеличений.

14.4. Точный поиск образца: наивный и КМП

Ищем все позиции, где P длины M полностью совпадает с фрагментом текста T длины N. Наивно пробуем $N-M+1$ начал и сравниваем до M символов: $O(NM)$ в худшем случае при $M \leq N$. Например, длинный текст из a и образец a…ab заставляют многократно сравнивать длинный общий префикс. Если $M > N$, непустой образец не встречается.

КМП хранит q — сколько первых символов P совпало с суффиксом уже просмотренного текста. При несовпадении $q \leftarrow \pi[q-1]$, не сдвигая указатель текста назад. При совпадении увеличиваем q. Если $q = M$, выдаём начало $i-M+1$ и откатываем $q = \pi[M-1]$, чтобы найти перекрывающиеся вхождения.

```
q = 0
for i in 0..N-1:
    while q > 0 and P[q] != T[i]: q = pi[q-1]
    if P[q] == T[i]: q += 1
    if q == M:
        report(i-M+1)
        q = pi[M-1]
```

Код предполагает $M > 0$. Для пустого образца заранее задаём соглашение: обычно он встречается в $N+1$ промежутках, и этот случай обрабатывают отдельно. Предобработка $O(M)$, поиск $O(N+z)$, где $z \leq N$ для непустого образца; обычно всё пишут $O(N+M)$. Память $O(M)$, текст можно читать потоком, результаты выдавать по мере появления.

Пример. P=«aba», π=[0,0,1], T=«ababa». После символа i=2 получаем q=3, выдаём 0, откат q=1. Затем b→q=2, a→q=3, выдаём 2. Если после совпадения сбрасывать q в 0, второе перекрывающееся вхождение можно пропустить.

14.5. Как понимать рисунок «( )x( )y» и экономить память?

Рисунок без конкретной строки неоднозначен. Правильный общий ответ: пусть уже есть совпавшие префикс и суффикс длины j. Новый символ y сравниваем с символом P[j], который мог быть обозначен x. Если x=y, граница растёт до j+1. Если нет, переходим к границе длины $\pi[j-1]$ и снова сравниваем **тот же** y с новым кандидатом. Покажите цепочку $j = 3 \to 1 \to 0$ из «ababac» в §14.1 вместо угадывания смысла скобок.

Альтернативный способ КМП — вычислять π для P+# +T, где # отсутствует во входных алфавитах. Если хранить всю объединённую строку и π, память $O(N+M)$. Потоковый вариант выше хранит только P и π(P), то есть $O(M)$. Для больших алфавитов не обязательно строить полную таблицу переходов $M \cdot \sigma$; КМП с π её не требует.

15. Ахо–Корасик: бор, ссылки, автомат

Статус: построение бора и конечного автомата вручную, суффиксные ссылки и сложности — из таблицы. Неясное «найти из текста образцы» уточняется в §16.5.

15.1. Что индексирует бор и какую задачу решает Ахо–Корасик?

Ответ на экзамене. Есть заранее заданный набор образцов. Строим по ним бор: путь от корня задаёт префикс, ребро помечено символом, терминальная вершина хранит номера заканчивающихся образцов. Общие префиксы хранятся один раз. Всего не более L+1 вершин. Бор не следует путать с суффиксным бором одного текста.

Ахо–Корасик добавляет суффиксные ссылки и автоматные переходы, позволяя за один проход по тексту находить вхождения всех образцов. Текущее состояние — самый длинный суффикс прочитанного текста, который является префиксом хотя бы одного образца. Суффиксная ссылка fail(v) ведёт к самому длинному **собственному** суффиксу строки вершины v, присутствующему в боре. Корень ссылается на себя.

15.2. Как построить суффиксные ссылки без угадывания?

Обходим бор BFS по возрастанию глубины. У детей корня fail=root. Пусть ребро u --c→ v и fail(u) уже известна. Из fail(u) ищем переход по c; если его нет, идём дальше по суффиксным ссылкам, до корня. Найденная вершина — fail(v). При построении **полного автомата** недостающий go(u,c) наследуется от go(fail(u),c), а в корне отсутствующий переход ведёт в корень.

Нужно отличать реальные рёбра бора от добавленных переходов автомата: иначе при BFS можно по ошибке бесконечно обходить циклы автомата как детей.

15.3. Полный пример: he, she, his, hers

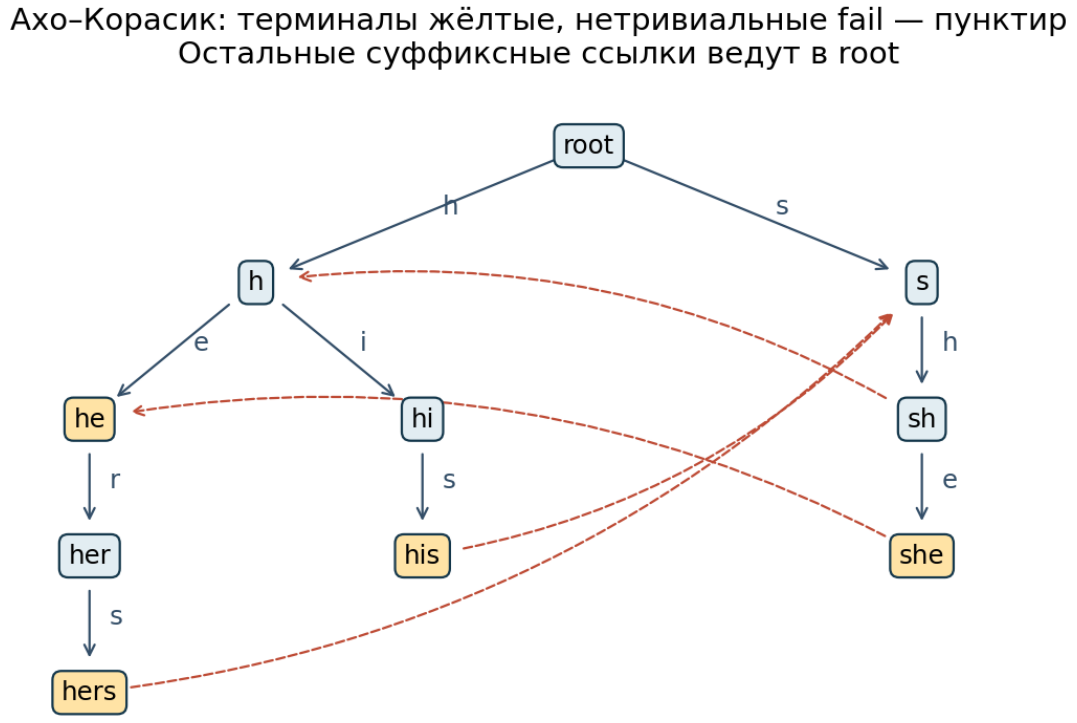
Рёбра бора: root→h→e→r→s; root→s→h→e; h→i→s. Терминальны вершины he, she, his, hers. Суффиксные ссылки:

Вершина (строка пути)	fail	Причина
h, s	root	собственный суффикс длины 0
he	root	e не является префиксом образца
hi	root	i не является префиксом
sh	h	h — самый длинный подходящий суффикс
her	root	r, er не являются префиксами
his	s	s — префикс she
she	he	he — префикс и целый образец
hers	s	s — подходящий суффикс

Прогон по «ushers» с индексами с нуля.

i	Символ	Состояние после символа	Выдача
0	u	root	нет
1	s	s	нет
2	h	sh	нет
3	e	she	she: [1,3]; по ссылке he: [2,3]
4	r	her через fail(she)=he	нет
5	s	hers	hers: [2,5]

Нельзя выдавать только образец в текущей вершине: на позиции 3 тогда потеряется he. Для эффективной выдачи храним выходную ссылку к ближайшей терминальной вершине по цепочке fail, а затем идём по выходным ссылкам; каждая такая остановка выдаёт реальное вхождение.



Бор he, she, his, hers с суффиксными ссылками

15.4. Почему поиск линейный и от чего зависит построение?

Пусть $V_{tr} \le L+1$. Для фиксированного небольшого алфавита с полной таблицей go время и память построения $O(V_{tr} \cdot \sigma + L)$; при постоянном σ это $O(L)$. Поиск $O(N+z)$: один переход по таблице на символ плюс работа пропорционально выдаче. Хранение только выходных ссылок занимает $O(V_{tr})$, в отличие от копирования всех списков ответов в каждую вершину, которое может раздуть память.

При разреженном хранении рёбер память бора $O(L)$, плюс словари и ссылки. Стоимость операций со словарём зависит от реализации: сбалансированное дерево даёт $O(\log \sigma)$, хеш-таблица — ожидаемое $O(1)$. Поиск с явными откатами по fail имеет суммарно $O(N)$ переходов при $O(1)$ доступе к ребру: глубина растёт не более чем на 1 за прочитанный символ и строго падает при откатах. Нельзя автоматически объявлять любую наивную разреженную реализацию построения $O(L)$: длинные отдельные поиски fail для многих рёбер требуют анализа, кеширования переходов либо выбранного линейного алгоритма.

Контроль вывода. Набор a,aa,…,a^k в тексте a^N порождает много совпадений. Время $O(N)$ без $+z$ для выдачи всех ответов физически невозможно. Повторяющиеся образцы могут хранить несколько номеров в одном терминале; пустые образцы требуют отдельного соглашения.

16. Суффиксные структуры и Укконен

Статус: назначение, построение на примере, память/время, выбор между массивом и деревом, сравнение с КМП и Ахо–Корасиком — из таблицы; Укконен — дополнение по программе.

16.1. Что такое суффиксный бор, сжатое дерево и массив?

Для текста T обычно добавляем уникальный терминатор $, отсутствующий в тексте. Он не должен считаться обычной буквой исходного поиска; для лексикографических примеров считаем его меньше всех остальных символов.

Суффиксный бор: в обычный бор вставлены все суффиксы T$. Наивная вставка суммарной длины $\Theta(N^2)$ требует $\Theta(N^2)$ времени и в худшем памяти. **Суффиксное дерево:** сжимаем каждый путь через вершины с одним ребёнком в одно ребро. Метка ребра хранится парой индексов [l,r) в исходном тексте, а не копией подстроки. Лист содержит начало соответствующего суффикса. В явном дереве O(N) вершин и рёбер; разветвления различаются первым символом ребра.

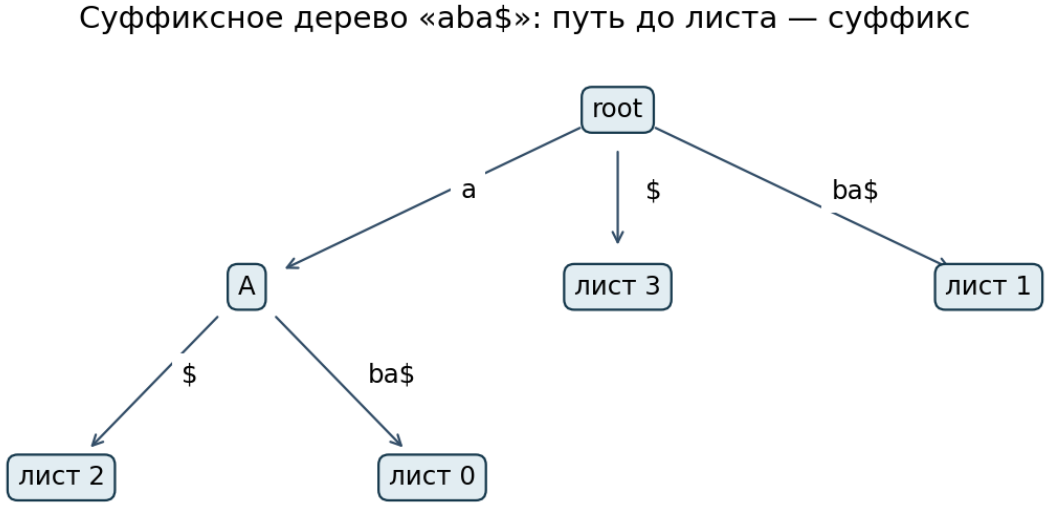
Суффиксный массив SA: массив начальных позиций всех суффиксов в лексикографическом порядке. Он занимает O(N) машинных слов и хранится вместе с текстом. Часто рядом хранят LCP — длины общих префиксов соседних в порядке SA суффиксов.

16.2. Как построить структуры для «aba$»?

Суффиксы: 0:aba$, 1:ba$, 2:a$, 3:$. Отсортированные: $, a$, aba$, ba$. Значит SA=[3,2,0,1]. При соглашении LCP[0]=0 и LCP[i]=lcp(SA[i−1],SA[i]) получим [0,0,1,0].

Дерево. Из корня три ребра: «$»→лист 3; «ba$»→лист 1; «a»→внутренняя вершина A. Из A: «$»→лист 2 и «ba$»→лист 0. Каждый путь от корня до листа — ровно один суффикс. Копировать «ba$» дважды в памяти не нужно: обе метки можно представить ссылками на текст.

Поиск «ba». В дереве выбираем из корня ребро, начинающееся b, и сравниваем первые два символа метки «ba$». Образец закончился внутри ребра — это успешный поиск; листья ниже дают позицию 1. В массиве бинарным поиском находим интервал суффиксов с префиксом ba; здесь это только SA[3]=1.



Сжатое суффиксное дерево для aba$

16.3. Сложности построения и поиска: какие именно?

Структура	Построение	Поиск образца длины M	Память индекса
Суффиксный бор	наивно O(N²)	O(M+z) при O(1) переходе	O(N²) в худшем
Суффиксное дерево	Укконен O(N) для фиксированного алфавита	O(M+z) при O(1) выборе ребра	O(N) слов
Суффиксный массив	удвоение с radix/counting sort O(N log N)	базово O(M log N+z)	O(N) слов
SA с LCP и подходящим поиском	дополнительная предобработка LCP/RMQ	возможно O(M+log N+z)	O(N) слов

Удвоение с обычной сравнительной сортировкой пар на каждом из O(log N) раундов даёт O(N log² N), а не O(N log N). Наивная сортировка суффиксов с посимвольным сравнением может стоить O(N²log N). Есть линейные алгоритмы SA для подходящей целочисленной модели алфавита, но это не свойство любой реализации.

При словарях с O(log σ) доступом переходы дерева добавляют соответствующий множитель; при хешировании линейность обычно ожидаемая. Если хранить таблицу σ переходов в каждой вершине, память становится O(Nσ), а не просто O(N) при растущем σ. Линейное дерево требует ссылок на метки рёбер; копирование всех меток уничтожает эту гарантию.

16.4. Укконен: идея, формализмы и небольшой прогон

Ответ на экзамене. Укконен строит суффиксное дерево онлайн, добавляя очередной символ текста и расширяя все нужные суффиксы. Он не проходит независимо каждый суффикс с нуля: использует общее окончание листьев, активную точку, суффиксные ссылки, правило ранней остановки и skip/count.

Что хранится. Рёбра — интервалы исходной строки. У всех листьев один изменяемый конец leaf_end: новая фаза удлиняет их сразу. Активная точка задаётся (active_node, active_edge, active_length), то есть вершиной, первым символом активного ребра и смещением внутри него. remainder — число ещё не выполненных явных расширений. Суффиксная ссылка внутренней вершины с меткой xα ведёт к вершине для α.

Правила расширения. Если в активной позиции нет нужного ребра, создаём лист. Если ребро есть и новый символ не совпадает в точке продолжения, расщепляем ребро и добавляем лист; новым внутренним вершинам ставим ссылки. Если нужный символ уже есть, увеличиваем active_length и останавливаем фазу: остальные суффиксы тоже уже представлены неявно (правило 3). После явного расширения уменьшаем remainder и сдвигаемся к следующему суффиксу через ссылку или специальное правило корня. Skip/count перепрыгивает всё ребро, если активная длина не меньше длины его метки.

Пример «aaa$».

Фаза	Что видно в неявном дереве	remainder в конце
a	корень — «a» — лист 0	0
aa	общий конец удлинил лист до «aa»; суффикс «a» оканчивается внутри ребра, правило 3	1
aaa	лист удлинился до «aaa»; «aa» и «a» пока неявны, правило 3	2
aaa$	$ не совпадает с a: последовательно расщепляем на глубинах 2 и 1, добавляем листья для суффиксов 1,2; добавляем корневой лист 3 для $	0

В последней фазе получается корневая ветка «a», затем ещё «a», затем из глубины 2 ветки «a$» (лист 0) и «$» (лист 1); из глубины 1 ещё ветка «$» (лист 2); из корня «$» (лист 3). Внутренние ссылки: вершина «aa»→«a»→root. Терминатор заставляет все оставшиеся суффиксы закончиться отдельными листьями.

Почему линейно. За весь алгоритм создаётся O(N) листьев и внутренних вершин. Успешные продолжения с остановкой случаются не более одного на фазу. Ссылки и skip/count не сравнивают длинные уже известные участки повторно; амортизированный учёт продвижений и переходов даёт O(N) для O(1) операций с исходящими рёбрами. Без skip/count и без суффиксных ссылок утверждение O(N) для произвольной реализации неверно. Для устного ответа важно объяснить, какую повторную работу устраняет каждый механизм, а не только назвать фамилию.

16.5. Что лучше: массив, дерево, КМП или Ахо–Корасик?

Главный вопрос — что многократно используется. Один заданный образец и один поток текста — КМП, без индекса всего текста. Много заранее заданных образцов и проход по тексту — Ахо–Корасик, индексируются образцы. Один неизменный текст и много будущих запросов — суффиксное дерево или массив, индексируется текст. Изменение набора образцов требует соответствующего обновления или перестроения Ахо–Корасика.

Дерево удобно для проходов по подстрокам и запросов через структуру ветвлений, базовый поиск O(M+z); массив компактен, последователен в памяти, удобен для SA/LCP-задач, часто проще в практическом хранении. У обоих линейное число слов при правильной реализации, но константы у дерева обычно больше.

Китайский против английского, повторы против разнообразия. По одному языку нельзя строго выбрать победителя. Большой σ делает плотные таблицы переходов дерева дорогими; разреженные словари снимают этот расход, но меняют время доступа. SA не хранит σ ячеек на вершину, поэтому часто практичен. Повторы меняют LCP, сжатие и константы, но не нарушают O(N) размер корректного суффиксного дерева. Утверждение из отзыва «массив всегда для разнообразных символов, дерево всегда для повторов» не является общей теоремой. Уточняем кодировку, Unicode-нормализацию, число запросов, память и реализацию.

Неоднозначный вопрос из допсессии. «Как найти из определённого текста с нуля образцы, которых n-ое количество» можно понять двояко. Если сами образцы даны — Ахо–Корасик. Если сначала дан текст, а образцы будут приходить позже — индексируем текст суффиксной структурой. Если требуется самостоятельно обнаружить интересные подстроки, нужен критерий «интересности» (повторяемость, длина и т. п.); это уже другая задача.

16.6. Пример полезного запроса по LCP

В строке «banana$» SA=[6,5,3,1,0,4,2], LCP=[0,0,1,3,0,0,2]. Максимальный LCP равен 3 и соответствует общей подстроке «ana» суффиксов с началами 3 и 1. Значит «ana» — самая длинная повторяющаяся подстрока (разрешаем перекрытия). В дереве ей соответствует самая глубокая внутренняя вершина по длине строки пути. Это пример, когда индекс текста полезнее повторных запусков КМП по всем возможным образцам.

17. Динамическое программирование

Статус: идея ДП, Фибоначчи с псевдокодом, мемоизация/табуляция и порядок перемножения матриц — из таблицы; НВП — дополнение по программе. TSP-ДП разобрано в §7.6, строковое ДП — в §13.

17.1. Как распознать задачу для ДП и доказать переход?

Ответ на экзамене. Выделяем состояние, полностью описывающее подзадачу; выражаем его ответ через меньшие состояния; задаём базы и порядок вычисления; доказываем, что рассмотрели все варианты последнего решения и не потеряли оптимум. Повторяющиеся подзадачи вычисляем один раз. Мемоизация — сверху вниз по запросу с кешем, табуляция — снизу вверх в заранее определённом порядке.

Шаблон: смысл D → допустимые состояния → база → переход → порядок → где ответ → восстановление → число состояний × стоимость перехода. Если неясно, что означает ячейка, формула сама по себе не является объяснением. Перекрывающиеся подзадачи мотивируют кеширование; оптимальная подструктура обосновывает оптимизационный переход. Для задач подсчёта и достижимости ДП тоже применимо.

17.2. Фибоначчи: рекурсия, мемоизация, табуляция

$F_0=0, F_1=1, F_n=F_{n-1}+F_{n-2}$, $n\geq 0$. Прямая рекурсия заново вычисляет одни и те же значения: например, F_5 вызывает F_4 и F_3, а F_4 снова вызывает F_3. Время $\Theta(\varphi^n)$, где $\varphi=(1+\sqrt{5})/2$; верхняя оценка $O(2^n)$ тоже верна, но грубее. Глубина стека $O(n)$.

Мемоизация хранит ответ каждого k: $O(n)$ арифметических операций и $O(n)$ памяти плюс стек. Табуляция:

```
fib(n):
    if n == 0: return 0
    a = 0; b = 1
    for k in 2..n:
        a, b = b, a+b   # одновременное присваивание
    return b
```

Инвариант. Перед итерацией k: $a=F_{k-2}$, $b=F_{k-1}$. После неё $a=F_{k-1}, b=F_k$. База k=2 верна, переход следует рекуррентной формуле, после k=n получаем ответ. При n=1 цикл пуст и возвращает 1. Последовательность до n=7: 0,1,1,2,3,5,8,13.

Время $O(n)$ сложений, память $O(1)$ машинных слов для фиксированной разрядности. Но точное F_n имеет $\Theta(n)$ бит: с длинной арифметикой обычная табуляция требует $O(n^2)$ битовых операций и $O(n)$ бит памяти для двух текущих чисел. Переполнение фиксированного целого не даёт математически точное F_n. Если спросят ускорение: быстрое удвоение или степень матрицы дают $O(\log n)$ арифметических шагов, но цена длинной арифметики остаётся.

17.3. Максимальная возрастающая подпоследовательность: состояние и граф

Ответ на экзамене. Подпоследовательность сохраняет порядок индексов, но не обязана быть непрерывной. Для строго возрастающей требуется $a[i_1]<a[i_2]<\dots$. D[i] — максимальная длина возрастающей подпоследовательности, оканчивающейся именно в i.

```
D[i] = 1 + max(D[j] по j<i и a[j]<a[i])
```

```
        # максимум пустого множества здесь считаем 0
  answer = max D[i]   # для пустой последовательности ответ 0
```

Храним predecessor[i], соответствующий выбранному j, и идём по нему назад для восстановления. Все j<i уже вычислены, значит время $O(n^2)$, память $O(n)$.

Графовое сведение. Создаём вершину для каждого индекса, дугу j→i при j<i и a[j]<a[i]. Это DAG, потому что индексы растут вдоль дуги. Возрастающая подпоследовательность — путь, ответ — самый длинный путь по числу вершин. Порядок индексов уже топологический. Для общего графа задача длиннейшего простого пути совсем другая; здесь решает ацикличность.

Пример. a=[3,1,2,5,4]. D=[1,1,2,3,3]. Для последнего 4 можно продолжить 1,2 до длины 3; 5 не подходит. Восстановление даёт [1,2,4] либо другой оптимум [1,2,5]. Нельзя требовать непрерывный отрезок: [1,2,4] пропускает 5.

17.4. Как получить НВП за O(n log n)?

Храним tails[k] — минимально возможный последний элемент возрастающей подпоследовательности длины k+1. Для очередного x бинарным поиском находим первую позицию со значением ≥x (lower_bound) и заменяем её на x; если такой позиции нет, дописываем x. Массив tails строго возрастает. Меньший хвост не хуже большего: его легче продолжить следующими элементами.

Пример [3,1,2,5,4]. tails: [3]→[1]→[1,2]→[1,2,5]→[1,2,4]. Ответ — длина 3. Но вообще tails как набор чисел не обязан сам быть подпоследовательностью исходного массива. Для восстановления храним индексы хвостов и предков при каждой замене. На каждый из n элементов O(log n), память O(n). Для неубывающей подпоследовательности используем первую позицию >x (upper_bound), а не ≥x; на [2,2] строгая длина 1, неубывающая — 2.

17.5. Порядок перемножения матриц: что оптимизируем?

Даны $A_1,\dots,A_k$ размеров $p_0\times p_1, p_1\times p_2,\dots,p_{k-1}\times p_k$. Их порядок менять нельзя; меняются только скобки, потому что умножение матриц ассоциативно, но не коммутативно. Обычное умножение (a×b) на (b×c) требует a·b·c скалярных умножений.

D[i,j] — минимум скалярных умножений для A_i…A_j. D[i,i]=0. Последнее умножение разбивает цепочку после t:

```
D[i,j] = min(t=i..j-1)
         (D[i,t] + D[t+1,j] + p[i-1]*p[t]*p[j]).
```

Считаем по возрастанию длины интервала; храним split[i,j] для восстановления скобок. **Доказательство:** последняя операция имеет некоторый t; обе её части должны быть вычислены оптимально, иначе улучшим всё выражение. Перебор всех t покрывает все варианты. $O(k^2)$ состояний, $O(k)$ разбиений в каждом → $O(k^3)$ времени, $O(k^2)$ памяти. Это затраты на выбор скобок, а не на реальное выполнение всех матричных умножений.

Полный пример. A:10×30, B:30×5, C:5×60. D[1,2]=10·30·5=1500, D[2,3]=30·5·60=9000. Для D[1,3]: t=1 даёт 0+9000+10·30·60=27000, t=2 даёт 1500+0+10·5·60=4500. Выбираем (AB)C, выигрыш в 6 раз. Результирующая матрица в обоих случаях 10×60.

18. Быстрое повторение и типичные ошибки

18.1. Сводка сложностей с условиями

Задача/алгоритм	Время	Дополнительная память / условие
3-раскраска: полный перебор	O(3^n(n+m))	O(n) сверх графа
Независимый класс + двудольность	O(2^n(n+m))	O(n) сверх графа
Класс размера ≤n/3	O*(1,8899^n)	полиномиальная
Случайные списки + 2-SAT	O((3/2)^n(n+m)ln(1/δ))	односторонняя ошибка ≤δ
Каргер: один запуск	$O(n^2)$	матричная реализация, $O(n^2)$
Каргер: ошибка ≤1/n	$O(n^4 \log n)$	$O(n^2 \log n)$ запусков
Каргер–Штейн: один запуск	$O(n^2 \log n)$	успех $\Omega(1/\log n)$
Каргер–Штейн: ошибка ≤1/n	$O(n^2 \log^3 n)$	повторения $O(\log^2 n)$
TSP: прямой перебор	O(n(n−1)!)	O(n) сверх графа
TSP: Held–Karp	$O(n^2 2^n)$	O(n2^n)
Метрический TSP, 2-приближение	$O(n^2)$ с Примом по матрице	неотрицательность, симметрия, треугольник
BFS	O(n+m)	O(n) сверх графа
Дейкстра / согласованный A*	O((n+m)log n)	куча; у A* ещё цена h
Форд–Фалкерсон	O((n+m)F)	целые ёмкости; O(n+m) память
Эдмондс–Карп	$O(nm^2)$	O(n+m)
Общий push–relabel	$O(n^2 m)$	O(n+m)
Левенштейн	O(len(A)·len(B))	две строки O(min(len(A),len(B))); путь — отдельно
КМП	O(N+M)	O(M), непустой образец
Ахо–Корасик	O(Lσ+N+z)	полные переходы, O(Lσ)
Суффиксное дерево	O(N) построение, O(M+z) поиск	постоянный алфавит, O(N) слов
Суффиксный массив удвоением	O(N log N), O(M log N+z) поиск	radix sort, O(N) слов
Фибоначчи, табуляция	O(n) сложений	O(1) слов; битовая цена в §17.2
НВП	$O(n^2)$ или O(n log n)	O(n)
Скобки матриц	$O(k^3)$	$O(k^2)$

18.2. Пятнадцать вопросов для самопроверки

Попробуйте ответить вслух, затем сверьтесь с ключом.

1. Почему K_5 не 3-раскрашиваем и какие именно множества проверит алгоритм $\le n/3$?
2. Что не так с проверкой только двудольности остатка без проверки S?
3. Почему вероятность успеха случайных списков $\ge (2/3)^n$, а не $(1/3)^n$?
4. Как из $(a \vee \neg b)$ построить две импликации?
5. Почему Каргер должен сохранять параллельные рёбра?
6. Что означает $n^2 \log n$: время одного Каргера или число запусков?
7. Можно ли остановить Дейкстру сразу при добавлении цели в очередь?
8. Почему A* на евклидовой сетке и Theta* могут дать разные длины?
9. Как отменить неудачный выбор пути в Форд–Фалкерсоне?
10. Достаточно ли отсутствия s–t-пути для готовности предпотока?
11. Где в 2-приближении TSP используется неравенство треугольника?
12. Почему у отжига нет универсальной оценки $O(n^2)$?
13. Почему откаты префикс-функции суммарно линейны?
14. Как на «ushers» не потерять вхождение he?
15. Какие два разбиения надо сравнить для матриц $10\times30, 30\times5, 5\times60$?

Ключ. 1: $\chi(K_5)=5$, проверяются $\varnothing$ и пять одиночных. 2: внутри S могут быть рёбра, дающие одинаковый первый цвет. 3: из трёх запрещаемых цветов два не совпадают с фиксированным правильным. 4: $\neg a \to \neg b$ и $b \to a$. 5: кратность влияет и на вес разреза, и на выборку. 6: это число независимых запусков для ошибки $\le 1/n$ при стандартной нижней оценке p. 7: нет, при актуальном минимальном извлечении. 8: разные множества разрешённых переходов. 9: пройти по обратной остаточной дуге и уменьшить исходный поток. 10: нет, промежуточные избытки должны исчезнуть. 11: при сокращении повторных посещений прямыми рёбрами. 12: неизвестны бюджет и стоимость хода. 13: каждое уменьшение j оплачивается предыдущими увеличениями. 14: пройти по выходной ссылке от she к he. 15: $A(BC)=27000$ и $(AB)C=4500$.

18.3. Что рисовать и говорить на практике

Раскраска: подпишите вершины; перечислите кандидаты; проверьте независимость; нарисуйте две доли остатка или конкретное нечётное кольцо. **Каргер:** подпишите состав супервершин и кратность рёбер после каждого стягивания. **Поток:** на каждой дуге пара «поток/ёмкость», обратные остаточные дуги отдельным стилем. **КМП:** индекс, символ, длина границы, цепочка откатов. **Ахо–Корасик:** реальные рёбра, fail, терминалы и выдача на каждой позиции. **Левенштейн:** подписи префиксов, нулевая строка и столбец, минимум трёх кандидатов, затем обратный путь. **Матричное ДП:** размеры матриц и стоимость каждого возможного последнего умножения.

Если исходная запись говорит только «показать на его словах» или «на его графе», конкретного исходного примера в файле нет. Используйте общий алгоритм и переносите его на выданные данные, не заучивайте учебные числа как единственный ответ.

Приложение А. Указатель исходных записей

Обработаны все четыре листа, включая содержательные примечания. Ниже 49 ячеек с учебными вопросами или общими указаниями на темы. В одной ячейке бывает несколько вопросов; повторы объединены в тексте. Это не статистика частоты экзаменационных билетов.

A.1. Лист «06.06.лист1»

Ячейка	Вопросы и уточнения в нормализованной форме	Где разобрано
C3	Раскраски; Каргер; Монте-Карло; изоморфизм	§3; 5; 6; 10
F3	Левенштейн: назначение, таблица, ответ, обратный ход; префикс-функция; КМП; Ахо–Корасик; суффиксные деревья; формула раскраски; повторения Каргера	§5.6; 6.3–6.4; 13–16
C4	Определение графа; 3-раскраска, множества на примере и применение; TSP	§1.3; 5.1–5.5; 7
C5	Графы и их виды; изоморфизм; 3-раскраска	§1.3; 5; 10
D5	Общие идеи строковых алгоритмов и сложности; конкретные названия не уточнены	§13–16; 18.1
C7	Все четыре алгоритма 3-раскраски и их сложности	§5.1–5.8
D7	Ахо–Корасик; КМП; Левенштейн; бор; построение суффиксного дерева; сложности	§13–16
C15	Каргер; Каргер–Штейн; динамическое программирование; TSP	§6; 7; 17
D15	Все строки; зачем суффиксные структуры и чем отличаются от КМП/Ахо–Корасика; автомат вручную; Левенштейн	§13–16, особенно 15.3; 16.5
C19	Разрезы; потоки; обратная дуга; ИИ и A*	§6.1; 9.1–9.3; 11
D19	Все строковые темы без конкретизации	§13–16
C21	Раскраски: малые множества, «пентаграмма», SAT→2-SAT; ДП для TSP; Фибоначчи, табуляция и мемоизация; локальный поиск, отжиг; площадь Монте-Карло	§3.2; 5; 7.3–7.6; 17.1–17.2
D21	Левенштейн; КМП; Ахо–Корасик; продолжение префикс-функции на рисунке ( )x( )y	§13–15, особенно 14.5
C22	Раскраска перебором множеств на нераскрашиваемом примере; обоснование сложности отжига	§5.3–5.4; 7.4
D22	КМП; Ахо–Корасик; суффиксные деревья; дополнительные вопросы без формулировок	§14–16
C30	TSP: перебор, локальный поиск; min-cut, потоки и связь разреза с потоком	§6.1; 7.1; 7.3; 9.1–9.3
D30	Сложности суффиксных структур; Левенштейн; вычисление префикс-функции	§13; 14.1–14.3; 16
C35	3-раскраска и K_5; инвариант push–relabel; применимость Монте-Карло; вероятность метода с 2-SAT; вероятность Каргера; A*, ИИ и Theta*	§3.1; 5; 6.3; 8.2–8.5; 9.4–9.5; 11
C56	Min-cut: сложность и применение; ошибки 1/e, 1/n; порог n/√2; условия Монте-Карло; Гольдберг, доказательство максимальности	§3.1; 3.4; 6; 9.4–9.6

Ячейка	Вопросы и уточнения в нормализованной форме	Где разобрано
D56	КМП: вся постановка, префикс-функция, сложности и линейность при откатах; суффиксный бор и дерево, построить для строки	§14; 16.1–16.4

A.2. Лист «06.06.лист2»

Ячейка	Вопросы и уточнения в нормализованной форме	Где разобрано
C6	Все варианты раскраски и TSP: идеи и обоснование времени	§5; 7
D6	Редакционное расстояние; построение бора; дерево против массива	§13; 15.1–15.3; 16
C10	Вероятностная 3-раскраска с обоснованием; Каргер и оптимизация	§5.6–5.8; 6
D10	Дерево против массива; отличие от Ахо–Корасика; Левенштейн; сложности построения	§13; 16.3–16.5
C12	Графы, цикл, дерево, гамильтонов и эйлеров циклы; TSP, локальный поиск на примере; прогон Дейкстры; изоморфизм	§1.3; 7.1–7.4; 8.1; 10
D12	Суффиксные массивы/деревья: мотивация, построение, поиск, выбор; автомат Ахо–Корасик	§15; 16
F12	Выбор дерева/массива по разнообразию символов, примеры китайского и английского: утверждение требует условий	§16.3; 16.5
C17	Стягивание ребра на примере; определение и пример изоморфизма	§6.2; 10.1
D17	Строки кроме Ахо–Корасика; происхождение сложностей; выбор дерева/массива по тексту	§13; 14; 16
F17	Определение «редукционного» предписания (уточнение термина: редакционное)	§13.1; 13.3
C18	Максимальный поток; Форд–Фалкерсон; все раскраски	§5; 9.1–9.3
D18	Префикс-функция; КМП; бор Ахо–Корасика и построение суффиксных ссылок	§14; 15
C23	Методы построения алгоритмов: backtracking, случайность, ветви/границы; успех Каргера; TSP, отсечки по пути/МОД; K_5 и n/3	§2; 3; 5.3–5.4; 6.3; 7.1–7.2
D23	Вычислить префикс-функцию; построить Ахо–Корасик; КМП и Левенштейн на заданных словах	§13–15
C37	Гольдберг; порядок перемножения матриц; граница TSP по МОД	§7.2; 9.4–9.6; 17.5
C38	Все TSP; K_5 перебором подмножеств; Монте-Карло для площади	§3.2; 5.4; 7
C41	Полные темы раскраски и TSP	§5; 7
D41	Строки: общие вопросы и сложности без конкретизации	§13–16; 18.1

A.3. Лист «06.06.лист3»

Ячейка	Вопросы и уточнения в нормализованной форме	Где разобрано
C7	TSP с анализом; локальный поиск; алгоритм разрезов и определение разреза	§6.1–6.4; 7.1–7.4
F7	Какие условия задают допустимый тур, помимо посещения вершин; гамильтонов цикл	§1.3; 7.1
C23	Стоимость стягивания; вероятность Каргера; число повторений для 1/e и 1/n; доказательство 2-приближения	§3.4; 6.2–6.4; 7.5
D23	КМП; Ахо–Корасик; суффиксные структуры, время/память/поиск и выбор; Левенштейн / Вагнер–Фишер	§13–16
C34	Каргер и Штейн, время и порог n/√2; ALT; условия применения Монте-Карло	§3.1; 6; 8.6
D34	Левенштейн, КМП и Ахо–Корасик с построением таблицы/π/бора/автомата	§13–15
C39	Все 3-раскраски; линейность 2-SAT; 2-приближение TSP и сложность	§5; 7.5
D39	Левенштейн и Ахо–Корасик: определение, таблица/автомат, сложность	§13; 15

A.4. Лист «допсессия 05.10 - 11.10»

Ячейка	Вопросы и уточнения в нормализованной форме	Где разобрано
D3	КМП; Вагнер–Фишер; наивный поиск; префикс-функция; редакционное расстояние; несколько образцов — неоднозначная постановка	§13–16, особенно 16.5
C4	Все 3-раскраски, подробно n/3; 2-приближённый TSP	§5; 7.5
D4	КМП; Вагнер–Фишер; суффиксные структуры и дерево	§13; 14; 16

A.5. Неполные записи и исключения

«собственный» (лист 1, D4), «Какие алгоритмы решают» (лист 2, C32) не позволяют восстановить задачу. «Не дошел до графов» не содержит вопроса. Общие отзывы «всё по строкам» учтены как ссылки на весь строковый раздел, без выдумывания уточнений. Бытовые реплики, сведения о студентах, оценки, время, декоративные подписи и мемы не являются вопросами.

Приложение Б. Навигация по дополнительным темам программы

Тема программы, не выделенная конкретным вопросом в таблице	Раздел
Виды сложности; vector и unordered_set	§1.1–1.2
Ферзи и оптимизации backtracking	§2.1
Жадность: самостоятельный МОД, расписание, Хаффман	§4
Подробный BFS, перевзвешивание и выбор эвристик A*	§8.1–8.4
Три направления ИИ и A*, названные в программе	§11.2–11.4
Динамические графы, обновление МОД	§12
Укконен	§16.4
НВП: ДП, DAG, O(n log n)	§17.3–17.4

Приложение B. Литература для сверки и углубления

Основные определения и доказательства изложены в самом справочнике. Ниже ориентиры для дополнительного чтения, а не утверждение о содержании конкретных лекций преподавателя.

1. Cormen, Leiserson, Rivest, Stein. Introduction to Algorithms: анализ, жадность, ДП, потоки, строки.
2. Karger, Stein. A New Approach to the Minimum Cut Problem (1996): рекурсивное случайное стягивание.
3. Goldberg, Tarjan. A New Approach to the Maximum-Flow Problem (1988): предпотоки и высоты.
4. Knuth, Morris, Pratt. Fast Pattern Matching in Strings (1977): линейный поиск образца.
5. Aho, Corasick. Efficient String Matching: An Aid to Bibliographic Search (1975): множественный поиск.
6. Ukkonen. On-line Construction of Suffix Trees (1995): активная точка и онлайн-построение.
7. Wagner, Fischer. The String-to-String Correction Problem (1974): редакционное расстояние.
8. Nash, Daniel, Koenig, Felner. Theta*: Any-Angle Path Planning on Grids (2007).
9. Goldberg, Harrelson. Computing the Shortest Path: A* Search Meets Graph Theory (2005): ALT.
10. Ullmann. An Algorithm for Subgraph Isomorphism (1976): матрица кандидатов и уточнение.